Threat Model

Threat Model - Juice Shop

Generated by appsec-advisor v0.4.0-beta (analysis v2)

Project	Juice Shop v19.2.1
Description	Probably the most modern and sophisticated insecure web application
Author	Björn Kimminich bjoern.kimminich@owasp.org (https://kimminich.de)
License	MIT
Repository	https://github.com/juice-shop/juice-shop
Homepage	https://owasp-juice.shop
Runtime	Node.js 20 - 24, Express 4
Tags	web security, web application security, webappsec, owasp, pentest, pentesting, security, vulnerable, vulnerability, broken, bodgeit, ctf, capture the flag, awareness

Changelog

Append-only history of assessment runs. Most recent first.

Version	Date	Mode	Depth	Reasoning	Baseline → Current	Δ Threats	Code	Note
v1	2026-05-30	full	standard	haiku-economy	(initial)	+31 / ~0 / -0	-	-

Table of Contents

Management Summary

Verdict

 Not production-ready. Juice Shop is a deliberate training target - every tier contains working exploits. Anyone with a browser and a clone of the public repo can take over admin accounts, execute server-side code, and read every user's credentials through several independent, unauthenticated paths. Fixing this requires rebuilding authentication, data access, and secret management - not patching a single bug.

Risk distribution: Critical: 9 · High: 16 · Medium: 6 · Low: 0 · **Total: 31**

Worst-case scenarios behind this verdict - what an attacker can do today:

- **Admin account takeover without any password** — The RSA signing key committed to the public repository lets anyone forge an admin token offline in under a minute, without touching the server. (F-002 — RSA private key hardcoded offline JWT forgery, F-001 — JWT algorithm confusion algorithm none bypass)
- **Full database read via the login form** — Typing a single character into the login email field bypasses authentication and returns the admin account; a follow-up payload dumps the full user table including all password hashes. (F-003 — SQL injection login authentication bypass, F-004 — SQL injection product search (UNION + schema exfil))
- **Arbitrary code runs on the server from a user profile update** — Setting a username to a JavaScript expression executes it in the server process, giving any registered user full control of the host operating system. (F-006 — Server side code injection via eval on username field, F-005 — Remote code execution notevil sandbox bypass in B2B API)
- **Every user password recoverable from a single database dump** — Passwords are stored as unsalted MD5 hashes — a format that maps to plaintext in seconds using widely available lookup tables. (F-027 — MD5 password hashing passwords trivially reversible)
- **Stored script in feedback panel hijacks admin sessions** — An attacker who injects a script into the feedback form has it execute in the admin's browser, stealing the session token stored in browser local storage. (F-020 — Stored XSS feedback comments rendered unsanitized in admin panel, F-019 — DOM XSS search results rendered via trust HTML bypass)

31 mitigations follow; moving secrets to runtime injection, switching raw SQL to parameterized queries, removing both eval sinks, and replacing MD5 password hashes are the four changes required before any production-readiness evaluation is meaningful.

Security Posture & Top Threats

Figure 1 - Architecture, Trust Boundaries & Threats

Components grouped by trust boundary (architecture tier). Grey edges are the legitimate request backbone. **Solid red** edges are attacker-controlled attacks - each originates at a threat actor and lands on the component it directly reaches. **Dotted red** edges are the consequence path: how that attack propagates through the application tier onto the data tier or the victim. Each red edge carries the threat glyph(s) - the same numbering as Figure 2 and the Top Threats table below.

Figure 2 - Risk Flow: Actor → Tier → Impact

Heatmap: **actors** (left) → **architecture tiers** (middle, Client → Application → Data) → **impact** (right). Numbered red arrows ①–⑥ are the threats enumerated in the Top Threats table below.

```
%%{init: {"flowchart": {"defaultRenderer": "elk", "nodeSpacing": 45, "rankSpacing": 95}} }%%
flowchart LR
    subgraph ACTORS[" "]
        direction TB
        HDR_A["<b>Threat Actors</b>"]:::columnHeader
        ANON(["fa:fa-user-secret Anonymous Internet Attacker"]):::actorAnon
        INTERNET_USER(["fa:fa-user-secret Authenticated Internet Attacker"]):::actorAnon
    end

    subgraph TIERS[" "]
        direction TB
        HDR_T["<b>Architecture Tiers</b>"]:::columnHeader
        BROWSER["Client Tier<br/>C-02 Angular Single-Page Application"]:::tierClient
        SERVER["Application Tier<br/>C-01 Express REST API Backend · C-04 File Upload and Processing
Service · C-05 B2B Order API (RCE Surface)"]:::tierApp
        DATA["Data Tier<br/>C-03 Data Layer (SQLite + MarsDB)"]:::tierData
    end

    subgraph IMPACT[" "]
        direction TB
        HDR_I["<b>Business Impact</b>"]:::columnHeader
        CUSTOMER_SESSION_HIJACK[["□ Customer Session Hijack"]]:::impact
        FULL_ADMIN_TAKEOVER[["□ Full Admin Takeover"]]:::impact
        FULL_SERVER_COMPROMISE[["□ Full Server Compromise"]]:::impact
        CUSTOMER_DATA_EXFILTRATION[["□ Customer Data Exfiltration"]]:::impact
    end

    %% Invisible alignment hints. Two purposes:
    %%   (a) keep per-component alignment edges (e.g. ANON --- SERVER) so the
    %%       forward attack arrows route horizontally without crossing tier
    %%       boxes (the diagram is forward-only: actor → tier → impact, no
    %%       backward relay edges into the actor column);
    %%   (b) chain the three column headers (HDR_A, HDR_T, HDR_I) across
    %%       subgraph boundaries so ELK pins them on the same Y line.
    HDR_A --- HDR_T
    HDR_T --- HDR_I
    ANON --- SERVER

    %% Attack arrows
    ANON ==>|" ① Injection<br/>② Auth Bypass<br/>④ Secret Exposure<br/>⑤ RCE  "| SERVER
    INTERNET_USER ==>|" ③ Priv-Esc  "| SERVER
    ANON ==>|" ⑥ XSS  "| BROWSER

    %% Consequence arrows (tier → business impact, all LR-forward)
    SERVER -.-> FULL_ADMIN_TAKEOVER
    SERVER -.-> CUSTOMER_DATA_EXFILTRATION
    SERVER -.-> FULL_SERVER_COMPROMISE
    BROWSER -.-> CUSTOMER_SESSION_HIJACK
    BROWSER -.-> FULL_ADMIN_TAKEOVER

    %% Subgraph frames invisible - column headers are emitted as the
    %% FIRST node of each subgraph (HDR_A / HDR_T / HDR_I) and pinned
    %% on the same Y line by the cross-subgraph alignment edges above.
    style ACTORS fill:none,stroke:none
    style TIERS  fill:none,stroke:none
    style IMPACT fill:none,stroke:none
```

```
    classDef tierClient fill:#f2f2f2,stroke:#424242,color:#111,stroke-width:2px,font-size:12px
    classDef tierApp    fill:#f2f2f2,stroke:#424242,color:#111,stroke-width:2px,font-size:12px
    classDef tierData   fill:#f2f2f2,stroke:#424242,color:#111,stroke-width:2px,font-size:12px

    classDef actorAnon     fill:#f3dada,stroke:#b71c1c,color:#7f0000,stroke-width:2px,font-size:12px
    classDef actorShopUser fill:#e8f1ea,stroke:#2e7d32,color:#1b5e20,stroke-width:2px,font-size:12px
    classDef impact        fill:#0f172a,stroke:#000,color:#fff,stroke-width:3px,font-size:12px

    classDef columnHeader  fill:none,stroke:none,color:#111,font-size:14px

    linkStyle 0,1,2     stroke:transparent,stroke-width:0px
    linkStyle 3,4,5     stroke:#b71c1c,stroke-width:3px
    linkStyle 6,7,8,9,10     stroke:#6b7280,stroke-width:1.5px,stroke-dasharray:4
```

Threat actors. The actors below drive the numbered attack paths in the figures above; the Shop User is the victim of client-side attacks (XSS / CSRF), not an attacker.

- **Shop User** — legitimate customer; target of client-side attacks; target of ⑥ Output Encoding / Cross-Site Scripting.
- **Anonymous Internet Attacker** — no account; registers in seconds when needed; drives ① Insecure Query Construction & Data Access, ② Hardcoded Secrets & Weak Cryptography, ④ Sensitive File & Secret Exposure, ⑤ Remote Code Execution (unsafe eval).
- **Authenticated Internet Attacker** — owns a regular account; logged in; drives ③ Broken Authorization & Access Control.

6 structural threats, grouped by weakness class - each row is one threat, not one finding. Threat Description states the general architectural weakness (STRIDE in brackets); Findings lists the concrete instances, each linked to §8 Threat Register with its component; Risk & Impact combines severity with business consequence.

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
①	**Insecure Query Construction & Data Access** (T·I) Untrusted input is concatenated directly into SQL and NoSQL query strings, allowing authentication bypass, data exfiltration, and schema disclosure through unauthenticated endpoints.	• F-003 — SQL injection login authentication bypass SQL injection login authentication bypass → C-01 • F-004 — SQL injection product search (UNION + schema exfil) SQL injection product search (UNION + schema exfil) → C-01 • F-026 — NoSQL injection via $where operator in MarsDB reviews NoSQL injection via $where operator in MarsDB reviews → C-03 • F-030 — Database schema exposure via SQL injection in search route Database schema exposure via SQL injection in search route → C-03	**Critical** Full Admin Takeover · Customer Data Exfiltration	M-003 — Replace raw SQL with Sequelize parameterized query, M-004 (P1)
②	**Hardcoded Secrets & Weak Cryptography** (S·E) Authentication can be circumvented by exploiting a hardcoded RSA signing key or the JWT algorithm-none bypass, allowing arbitrary token forgery without any server interaction.	• F-001 — JWT algorithm confusion algorithm none bypass JWT algorithm confusion algorithm none bypass → C-01 • F-002 — RSA private key hardcoded offline JWT forgery RSA private key hardcoded offline JWT forgery → C-01 • F-027 — MD5 password hashing passwords trivially reversible MD5 password hashing passwords trivially reversible → C-01	**Critical** Full Admin Takeover · Customer Data Exfiltration	M-002 — Implement application-level CSP and remove bypassSecurityTrustHtml, M-027 (P1)
③	**Broken Authorization & Access Control** (E·I) Authorization checks on resource-owning endpoints accept attacker-controlled user identifiers from the request body rather than the verified JWT payload, enabling horizontal and vertical access escalation.	• F-011 — IDOR basket accessible by any authenticated user IDOR basket accessible by any authenticated user → C-01 • F-017 — Mass assignment wallet balance manipulable via req.body.UserId Mass assignment wallet balance manipulable via req.body.UserId → C-01 • F-023 — Angular route guards bypassable client side Angular route guards bypassable client side → C-02	**High** Full Admin Takeover · Customer Data Exfiltration	M-011 — Validate basket ownership against authenticated user's ID from JWT, M-017 (P2)

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
		• F-029 — Product reviews allow unauthorized update IDOR in review update Product reviews allow unauthorized update IDOR in review update → C-03		
④	**Sensitive File & Secret Exposure** (I) Signing keys, logs, and metric endpoints are served over unauthenticated routes, and path-traversal weaknesses allow reading arbitrary files from the server filesystem.	• F-010 — Sensitive files and logs exposed without authentication Sensitive files and logs exposed without authentication → C-01 • F-013 — Prometheus metrics endpoint exposed without authentication Prometheus metrics endpoint exposed without authentication → C-01 • F-014 — Access logs exposed to unauthenticated users tamper potential Access logs exposed to unauthenticated users tamper potential → C-01 • F-015 — Path traversal in data erasure layout parameter Path traversal in data erasure layout parameter → C-01 • F-028 — User model returns sensitive fields in API responses (password hash, totpSecret) User model returns sensitive fields in API responses (password hash, totpSecret) → C-03	**High** Customer Data Exfiltration · Full Server Compromise	M-010 — Add authentication middleware to sensitive file serving routes, M-015 (P2)
⑤	**Remote Code Execution (unsafe eval)** (E) Attacker-controlled strings are evaluated as JavaScript code through two independent sinks - a profile-field handler and a B2B order endpoint - providing direct host-level code execution.	• F-005 — Remote code execution notevil sandbox bypass in B2B API Remote code execution notevil sandbox bypass in B2B API → C-01 • F-006 — Server side code injection via eval on username field Server side code injection via eval on username field → C-01	**Critical** Full Server Compromise · Customer Data Exfiltration	M-005 — Remove eval-based order processing; use structured input schema, M-006 (P1)
⑥	**Output Encoding / Cross-Site Scripting**	• F-019 — DOM XSS search results rendered via trust	**Critical** Customer Session	M-019 — Remove

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
	(T·I) Angular's default output encoding is bypassed via DomSanitizer trust overrides across 14+ components, enabling stored and DOM-based XSS that steals session tokens from browser localStorage.	HTML bypass DOM XSS search results rendered via trust HTML bypass → C-02 • F-020 — Stored XSS feedback comments rendered unsanitized in admin panel Stored XSS feedback comments rendered unsanitized in admin panel → C-02 • F-021 — XSS last login ip rendered as trusted HTML XSS last login ip rendered as trusted HTML → C-02 • F-025 — Product description and review rendered as HTML stored XSS vector Product description and review rendered as HTML stored XSS vector → C-02	Hijack · Full Admin Takeover	bypassSecurityTrustHtml ; use Angular's default sanitization, M-020 (P1)

STRIDE: S spoofing · T tampering · R repudiation · I information disclosure · D denial of service · E elevation of privilege. Risk, findings, components, impact and Fix are derived deterministically; only the one-line weakness description is authored.

Top Mitigations

Highest-impact P1/P2 mitigations - 10 of 25 qualifying (31 total). Full detail in §9 Mitigation Register. All 9 mitigation(s) that fix a Critical finding are always listed here; the remaining entries are curated by impact: broad SQL injection, stored XSS, SSRF, and IDOR exposure that combines with the hardcoded secrets to produce the most-reachable attack chains

#	Priority	Component	Mitigation	Addresses	Effort
1	P1	C-01 — Express REST API Backend	M-003 — Replace raw SQL with Sequelize parameterized query	F-003 — SQL injection login authentication bypass (`routes/login.ts`)	Low
2	P1	C-01 — Express REST API Backend	M-004 — Use Sequelize Op.like with parameterized values for search	F-004 — SQL injection product search (`routes/search.ts` , "UNION + schema exfil")	Low
3	P1	C-01 — Express REST API Backend	M-006 — Remove `eval()` from username processing; use safe string rendering	F-006 — Server side code injection via eval on username field (`routes/userProfile.ts`)	Low
4	P1	C-01 — Express REST API Backend	M-002 — Implement application-level CSP and remove bypassSecurityTrustHtml	F-002 — RSA private key hardcoded offline JWT forgery (`lib/insecurity.ts`)	Medium
5	P1	C-01 — Express REST API Backend	M-005 — Remove eval-based order processing; use structured input schema	F-005 — Remote code execution notevil sandbox bypass in B2B API (`routes/b2bOrder.ts`)	Medium
6	P1	C-01 — Express REST API Backend	M-027 — Replace MD5 with bcrypt (cost factor 12+) for password hashing	F-027 — MD5 password hashing passwords trivially reversible (`lib/insecurity.ts`)	Medium
7	P1	C-02 — Angular Single-Page Application	M-019 — Remove bypassSecurityTrustHtml; use Angular's default sanitization	F-019 — DOM XSS search results rendered via trust HTML bypass (`frontend/src/app/search-result/search-result.component.ts`)	Low
8	P1	C-02 — Angular Single-Page Application	M-020 — Sanitize feedback content server-side before storage; remove bypassSecurityTrustHtml in admin panel	F-020 — Stored XSS feedback comments rendered unsanitized in admin panel (`frontend/src/app/administration/administration.component.ts`)	Medium
9	P2				Me

#	Priority	Component	Mitigation	Addresses	Effort
		C-01 — Express REST API Backend	M-001 — Establish dependency vulnerability scanning and update SLA	F-001 — JWT algorithm confusion algorithm none bypass (`lib/insecurity.ts`)	dium
10	P2	C-01 — Express REST API Backend	M-007 — Set `noent:false` in libxmljs2 parsing to disable external entity resolution	F-007 — XXE via XML file upload with external entity resolution (`routes/fileUpload.ts`)	Low

15 additional P1/P2 mitigations capped from the leader-board · 6 P3 backlog items in §9 Mitigation Register. Sorted by priority (P1 first), then component, then leverage (most findings first), severity (Critical first), and effort (Low first).

Operational Strengths

Operational controls rated Adequate or Partial - grouped into broad clusters (full per-control breakdown in §7). Clusters demoted to Weak by open Critical/High findings appear in §7 instead, not here.

Strength	What's in Place	Effectiveness	Gap	Mitigates
Container & Supply-Chain Hardening	Build-time and runtime hardening - minimal base image, non-root execution, dependency inventory. Automated SCA scanning Container Security - Dockerfile:23-41 File Upload Validation - `routes/fileUpload.ts`, `routes/profileImageUrlUpload.ts`	□ Adequate	-	-
Observability & Audit	Runtime visibility - access logging, audit trails, and operational telemetry for post-incident review. Security Logging and Monitoring - `server.ts:338`, `lib/logger.ts`	⚠ Partial	Coverage incomplete - see §7 control assessment.	-

Bottom line: These controls narrow specific attack surfaces but none eliminates a Critical finding on its own.

Critical Attack Tree

The root is the worst-case attacker goal; below it, each capability branch groups the Critical findings that achieve it. Branches feed the goal by OR - any single path suffices.

```
graph LR
    G_ROOT["Full host and data compromise"]:::goal
    OR_ROOT["Any one path suffices"]:::or_node
    AND_JWT["Forge admin JWT offline"]:::and_node
    L_T001["T-001 - JWT algorithm confusion bypass"]:::leaf
    L_T002["T-002 - RSA private key hardcoded"]:::leaf
    OR_RCE["Server-side code execution"]:::or_node
    L_T005["T-005 - RCE notevil sandbox bypass"]:::leaf
    L_T006["T-006 - eval on username field"]:::leaf
    OR_SQLI["Authentication bypass via SQL"]:::or_node
    L_T003["T-003 - SQL injection login"]:::leaf
    L_T004["T-004 - SQL injection search UNION"]:::leaf
    AND_XSS["Session hijack via XSS + localStorage"]:::and_node
    L_T019["T-019 - DOM XSS search results"]:::leaf
    L_T020["T-020 - Stored XSS feedback admin"]:::leaf
    L_T027["T-027 - MD5 password hashing"]:::leaf

    OR_ROOT -->|"OR"| G_ROOT
    AND_JWT -->|"OR"| OR_ROOT
    L_T001 -->|"AND"| AND_JWT
    L_T002 -->|"AND"| AND_JWT
    OR_RCE -->|"OR"| OR_ROOT
    L_T005 -->|"OR"| OR_RCE
    L_T006 -->|"OR"| OR_RCE
    OR_SQLI -->|"OR"| OR_ROOT
    L_T003 -->|"OR"| OR_SQLI
    L_T004 -->|"OR"| OR_SQLI
    AND_XSS -->|"OR"| OR_ROOT
    L_T019 -->|"AND"| AND_XSS
    L_T020 -->|"AND"| AND_XSS
    L_T027 -->|"AND"| AND_XSS

    classDef goal fill:#0f172a,stroke:#000,color:#fff,stroke-width:3px
    classDef and_node fill:#334155,stroke:#1e293b,color:#fff,stroke-width:2px
    classDef or_node fill:#64748b,stroke:#334155,color:#fff,stroke-width:2px
    classDef leaf fill:#f3dada,stroke:#b71c1c,color:#7f0000,stroke-width:2px
```

Findings (full detail in §8 Threat Register): F-001 — JWT algorithm confusion algorithm none bypass JWT algorithm confusion bypass · F-002 — RSA private key hardcoded offline JWT forgery RSA private key hardcoded · F-005 — Remote code execution notevil sandbox bypass in B2B API RCE notevil sandbox bypass · F-006 — Server side code injection via eval on username field eval on username field · F-003 — SQL injection login authentication bypass SQL injection login · F-004 — SQL injection product search (UNION + schema exfil) SQL injection search UNION · F-019 — DOM XSS search results rendered via trust HTML bypass DOM XSS search results · F-020 — Stored XSS feedback comments rendered unsanitized in admin panel Stored XSS feedback admin · F-027 — MD5 password hashing passwords trivially reversible MD5 password hashing

---

1. System Overview

Probably the most modern and sophisticated insecure web application

Repository: https://github.com/juice-shop/juice-shop **Runtime:** `Node.js` 20 - 24

Scope

This threat model covers 5 components of juice-shop: **Express REST API Backend**, **Angular Single-Page Application**, **Data Layer (SQLite + MarsDB)**, **File Upload and Processing Service**, **B2B Order API (RCE Surface)**.

Out of scope: third-party hosted dependencies, browser runtime, operating-system kernel, and the underlying network infrastructure.

---

2. Architecture Diagrams

2.1 System Context

Who interacts with juice-shop from the outside, and through which channels. Solid arrows show normal usage; dashed red arrows mark unauthenticated probing or exploit paths (C4 Level 1).

```
flowchart LR
    USER["End User<br/>(browser)"]
    ATTACKER["Anonymous<br/>Internet Attacker"]
    ADMIN["Admin User"]
    SYSTEM["juice-shop"]
    EXTERNAL["External HTTP Services<br/>(SSRF target)"]
    USER -->|HTTPS · normal usage| SYSTEM
    ATTACKER -.->|HTTPS · probing / exploit| SYSTEM
    ADMIN -->|HTTPS · admin actions| SYSTEM
    SYSTEM -->|outbound · HTTPS| EXTERNAL
    classDef user     fill:#e8f1ea,stroke:#2e7d32,color:#1b5e20,stroke-width:1.5px
    classDef attacker fill:#f3dada,stroke:#b71c1c,color:#7f0000,stroke-width:2px
    classDef admin    fill:#fef3c7,stroke:#b45309,color:#78350f,stroke-width:1.5px
    classDef sys      fill:#f2f2f2,stroke:#424242,color:#111,stroke-width:1.5px
    classDef ext      fill:#f2f2f2,stroke:#9e9e9e,color:#424242,stroke-dasharray:3 3,stroke-width:1px
    class USER user
    class ATTACKER attacker
    class ADMIN admin
    class SYSTEM sys
    class EXTERNAL ext
```

2.2 Container Architecture

How the system decomposes into deployable units. Each box is a separate runtime process or service container; arrows show synchronous request paths between them. Components with ≥3 Critical findings carry a red border, ≥2 High amber (C4 Level 2).

```
flowchart TB
    subgraph Client
        frontend_spa["Angular Single-Page Application"]
    end
    subgraph Application
        backend_api["Express REST API Backend"]
        file_upload_service["File Upload and Processing Service"]
        b2b_api["B2B Order API (RCE Surface)"]
    end
    subgraph Data
        data_persistence[("Data Layer (SQLite + MarsDB)")]
    end
    frontend_spa -->|HTTPS REST| backend_api
    backend_api -->|driver| data_persistence
    backend_api -->|in-process| file_upload_service
    backend_api -->|in-process| b2b_api
    classDef critical fill:#f3dada,stroke:#b71c1c,color:#7f0000,stroke-width:3px
    classDef warning  fill:#fef3c7,stroke:#b45309,color:#78350f,stroke-width:2px
    class backend_api critical
    class frontend_spa warning
    class data_persistence warning
```

2.3 Components

Who reaches each component, and through which trust zone. Four columns map external actors to the internal tiers (Client / Application / Data); solid green arrows show legitimate data flow, dashed red arrows mark intrusion vectors. The component table directly below holds source paths and linked threats per `C-NN`; per-finding evidence is in §8 Threat Register.

```
flowchart TD
    subgraph EXT["Untrusted Zone - Internet"]
        INTERNET_ANON["fa:fa-user-secret Anonymous Internet Attacker"]:::threat
        VICTIM_REQUIRED["fa:fa-user Shop User"]:::legit
        REPO_READ["fa:fa-code-branch Internal Developer"]:::threat
    end
    subgraph CLIENT["Client Tier"]
        frontend_spa["fa:fa-window-restore frontend-spa Angular Single-Page Appli…<br/><i>7 threats</
i>"]:::risk
    end
    subgraph APP["Application Tier"]
        backend_api["fa:fa-server backend-api Express REST API Backend<br/>+ file-upload-service + b2b-
api<br/><i>19 threats</i>"]:::risk
    end
    subgraph DATA["Data Tier"]
        data_persistence[("fa:fa-database data-persistence Data Layer (SQLite + MarsDB)<br/><i>5
threats</i>")]:::risk
    end
    VICTIM_REQUIRED -->|"HTTPS · TLS"| frontend_spa
    frontend_spa -->|"REST · JWT Bearer"| backend_api
    backend_api -->|"ORM · queries"| data_persistence
    INTERNET_ANON -.->|"injection · auth bypass · RCE"| backend_api
    INTERNET_ANON -.->|"XSS · client tampering · token theft"| frontend_spa
    REPO_READ -.->|"leaked credentials · auth bypass"| backend_api

    classDef legit fill:#e8f1ea,stroke:#2e7d32,color:#1b5e20,stroke-width:1.5px
    classDef threat fill:#f3dada,stroke:#b71c1c,color:#7f0000,stroke-width:2px
    classDef external fill:#f2f2f2,stroke:#424242,color:#212121,stroke-width:1.5px
    classDef risk fill:#fef2f2,stroke:#991b1b,color:#111,stroke-width:2.5px
    linkStyle 0,1,2 stroke:#2e7d32,stroke-width:1.5px
    linkStyle 3,4,5 stroke:#b71c1c,stroke-width:2.5px,stroke-dasharray:6 4
```

ID	Name	Type	Key Paths	Linked Threats
C-01	Express REST API Backend	process	`server.ts` `routes/**` `lib/**` `app.ts`	F-001 (JWT algorithm confusion algorithm none bypass) F-002 (RSA private key hardcoded offline JWT forgery) F-003 (SQL injection login authentication bypass) F-004 (SQL injection product search) F-005 (Remote code execution notevil sandbox bypass in B2B API) F-006 (Server side code injection via eval on username field) F-007 (XXE via XML file upload with external entity resolution) F-008 (Zip slip path traversal via unzipper 0.9.15) F-009 (Server side request forgery via profile image URL upload) F-010 (Sensitive files and logs exposed without authentication) F-011 (IDOR basket accessible by any authenticated user) F-012 (Open redirect bypass via allowlist prefix check) F-013 (Prometheus metrics endpoint exposed without authentication) F-014 (Access logs exposed to unauthenticated users tamper potential) F-015 (Path traversal in data erasure layout parameter) F-016 (Missing rate limiting on authentication endpoint) F-017 (Mass assignment wallet balance manipulable via `req.body`.UserId) F-018 (Error handler reveals internal stack traces) F-027 (MD5 password hashing passwords trivially reversible)
C-02	Angular Single-Page Application	process	`frontend/src/**`	F-019 — DOM XSS search results rendered via trust HTML bypass F-020 — Stored XSS feedback comments rendered unsanitized in admin panel F-021 — XSS last login ip rendered as trusted HTML F-022 — Client side JWT stored in localStorage accessible to XSS F-023 — Angular route guards bypassable client side F-024 — XSS via postMessage and iframe

ID	Name	Type	Key Paths	Linked Threats
				DOM based attack F-025 — Product description and review rendered as HTML stored XSS vector

C-03	Data Layer (SQLite + MarsDB)	datastore	`models/**`
			`data/**`
			`routes/showProductReviews.ts`
			`routes/search.ts`
			`routes/updateProductReviews.ts`
				F-026 — NoSQL injection via $where operator in MarsDB reviews
				F-028 — User model returns sensitive fields in API responses (password hash, totpSecret)
				F-029 — Product reviews allow unauthorized update IDOR in review update
				F-030 — Database schema exposure via SQL injection in search route
				F-031 — JavaScript injection via $where can crash MarsDB process

C-04	File Upload and Processing Service	process	`routes/fileUpload.ts`
			`routes/profileImageFileUpload.ts`
			`routes/profileImageUrlUpload.ts`
			`routes/memory.ts`
				-

C-05	B2B Order API (RCE Surface)	gateway	`routes/b2bOrder.ts`	-

2.4 Technology Architecture

The technology stack the system is built on. Each box names the framework or runtime that fills that role; per-component findings live in the §2.3 component table above, and the full per-finding catalogue is in §8 Threat Register.

```
flowchart TD
    subgraph CLIENT["Client Tier"]
        FE_ANGULAR["fa:fa-window-restore Angular SPA<br/><i>browser runtime</i>"]:::risk
    end
    subgraph APP["Application Tier"]
        RUNTIME["fa:fa-server Node.js<br/><i>JS runtime</i>"]:::risk
        EXPRESS["fa:fa-server Express<br/><i>HTTP framework</i>"]:::risk
        AUTH_MW["fa:fa-shield-halved express-jwt · helmet · CORS<br/><i>auth middleware</i>"]:::risk
    end
    subgraph DATA["Data Tier"]
        ORM["fa:fa-database Sequelize ORM<br/><i>object-relational mapper</i>"]:::risk
        SQLITE[("fa:fa-database SQLite<br/><i>embedded relational DB</i>")]:::risk
        MARSDB[("fa:fa-database MarsDB<br/><i>in-memory NoSQL</i>")]:::risk
        LOCAL_FS["fa:fa-folder-open Local FS<br/><i>uploads · logs · keys</i>"]:::risk
    end
    subgraph INFRA["Cross-Cutting"]
        INFRA_SCM["fa:fa-code-branch GitHub (public)<br/><i>source supply chain</i>"]:::risk
    end
    FE_ANGULAR -->|"HTTPS · JWT"| AUTH_MW
    AUTH_MW -->|"middleware chain"| EXPRESS
    EXPRESS -->|"DB driver"| ORM
    EXPRESS -->|"DB driver"| SQLITE
    EXPRESS -->|"DB driver"| MARSDB
    EXPRESS -->|"file I/O"| LOCAL_FS
    INFRA_SCM -.->|"clone · extract secrets"| EXPRESS

    classDef risk fill:#fef2f2,stroke:#991b1b,color:#111,stroke-width:2.5px
    classDef ok fill:#e8f1ea,stroke:#2e7d32,color:#1b5e20,stroke-width:1.5px
    linkStyle 0,1,2,3,4,5 stroke:#424242,stroke-width:1.5px
    linkStyle 6 stroke:#9e9e9e,stroke-width:1px,stroke-dasharray:3 3
```

> **Legend: red border** ≥ 3 Critical threats on the component · **amber border** ≥ 2 High threats

---

3. Attack Walkthroughs

This section walks through how the highest-risk findings are exploited - one short walkthrough per Critical, each with attack steps, a focused sequence diagram, and the primary mitigation. The cross-finding view (which weaknesses combine toward the worst-case goal, and where one fix severs several paths) is in the Critical Attack Tree. Full per-finding context - severity rationale, assets, detection signals - is in the §8 Threat Register row for each finding.

3.1 JWT algorithm confusion algorithm none bypass

Source: F-001 — JWT algorithm confusion algorithm none bypass - `lib/insecurity.ts:54`

Severity **Critical** (CWE-347). STRIDE: Spoofing. See §8 T-001 for the full register row.

Attack Steps

1. An attacker crafts a JWT with header {"alg":"none"} and empty signature.

2. express-jwt 0.1.3 (routes via `lib/insecurity.ts:54`) accepts unsigned tokens because it does not enforce algorithm.
3. Attacker sets isAdmin:true in the payload and gains full admin access.

Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Attacker
    participant App
    Note over App: backend-api - lib/insecurity.ts:54
    Attacker->>App: Crafted request exercising the weakness at lib/insecurity.ts line 54
    App->>App: Vulnerable branch executes without the missing control
    App-->>Attacker: Response confirms the weakness - CWE-347
```

Defense in Depth

- Primary mitigation: M-001 (Establish dependency vulnerability scanning and update SLA)

3.2 RSA private key hardcoded offline JWT forgery

Source: F-002 — RSA private key hardcoded offline JWT forgery - `lib/insecurity.ts:23`

Severity **Critical** (CWE-321). STRIDE: Spoofing. See §8 T-002 for the full register row.

Attack Steps

1. The RSA 2048-bit private key is embedded in `lib/insecurity.ts:23` and committed to a public GitHub repository.
2. Any user can retrieve the key (git clone or GitHub web UI), call `jwt.sign`({id:1, email:'admin@juice-`sh.op`', isAdmin:true}, privateKey, {algorithm:'RS256'}) offline and produce a cryptographically valid admin JWT without any server interaction.
3. Clone the public repository and locate the cryptographic key at `lib/insecurity.ts:23`.

Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Attacker
    participant Repo
    participant App
    Note over Repo: Public repository
    Note over App: backend-api - lib/insecurity.ts:23
    Attacker->>Repo: Clone or browse - extract private key from lib/insecurity.ts line 23
    Attacker->>Attacker: Sign forged token / signature with the extracted key
    Attacker->>App: Submit forged artefact in normal authentication flow
    App-->>Attacker: Request accepted as authentic - signature validates
```

Defense in Depth

- Primary mitigation: M-002 (Implement application-level CSP and remove bypassSecurityTrustHtml)

3.3 SQL injection login authentication bypass

Source: F-003 — SQL injection login authentication bypass - `routes/login.ts:34`

Severity **Critical** (CWE-89). STRIDE: Tampering. See §8 T-003 for the full register row.

Attack Steps

1. `routes/login.ts:34` constructs an SQL query via template literal: SELECT * FROM Users WHERE email = '${`req.body`.email}' AND password = '...' AND deletedAt IS NULL.
2. An attacker submits email=admin@juice-`sh.op`'-- which comments out the password check, authenticating as admin without knowing the password.
3. UNION-based injection can dump the entire Users table.

Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Attacker
    participant API
    participant DB
    Note over API: backend-api - routes/login.ts:34
    Note over DB: Database
    Attacker->>API: Crafted HTTP request to the affected endpoint with classical OR 1=1 payload
    API->>DB: SQL built by string interpolation - payload becomes query
    DB-->>API: First matching row regardless of intended predicate
    API-->>Attacker: 200 OK with authenticated session / leaked rows
```

Defense in Depth

- Primary mitigation: M-003 (Replace raw SQL with Sequelize parameterized query)

3.4 SQL injection product search (UNION + schema exfil)

Source: F-004 — SQL injection product search (UNION + schema exfil) - `routes/search.ts:23`

Severity **Critical** (CWE-89). STRIDE: Tampering. See §8 T-004 for the full register row.

Attack Steps

1. `routes/search.ts:23` builds: SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' ...).
2. An attacker submits q=')) UNION SELECT * FROM Users-- to enumerate all users.
3. The same route at line 47 runs SELECT sql FROM sqlite_master on UNION success, exposing the full database schema.

Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Attacker
    participant API
    participant DB
    Note over API: backend-api - routes/search.ts:23
    Note over DB: Database
    Attacker->>API: Crafted HTTP request to the affected endpoint with classical OR 1=1 payload
    API->>DB: SQL built by string interpolation - payload becomes query
    DB-->>API: First matching row regardless of intended predicate
    API-->>Attacker: 200 OK with authenticated session / leaked rows
```

Defense in Depth

- Primary mitigation: M-004 (Use Sequelize `Op.like` with parameterized values for search)

3.5 Remote code execution notevil sandbox bypass in B2B API

Source: F-005 — Remote code execution notevil sandbox bypass in B2B API - `routes/b2bOrder.ts:23`

Severity **Critical** (CWE-94). STRIDE: Elevation of Privilege. See §8 T-005 for the full register row.

Attack Steps

1. POST `/b2b/v2/orders` accepts orderLinesData (user-controlled string) and executes it via vm.runInContext('safeEval(orderLinesData)', sandbox) at `routes/b2bOrder.ts:23`.
2. The notevil library has documented sandbox escape vulnerabilities.
3. An attacker can execute arbitrary `Node.js` code by sending a crafted payload: {"orderLinesData":"require('child_process').execSync('id')"}.

Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Attacker
    participant API
    participant Sandbox
    Note over API: backend-api - routes/b2bOrder.ts:23
    Note over Sandbox: Eval / VM context
    Attacker->>API: POST /b2b/v2/orders with JS payload escaping the sandbox
    API->>Sandbox: Pass attacker-controlled code to eval / vm.runInContext
    Sandbox->>Sandbox: Prototype-walk reaches the host realm - RCE
    API-->>Attacker: Command output / arbitrary side effects
```

Defense in Depth

- Primary mitigation: M-005 (Remove eval-based order processing; use structured input schema)

3.6 Server side code injection via eval on username field

Source: F-006 — Server side code injection via eval on username field - `routes/userProfile.ts:62`

Severity **Critical** (CWE-95). STRIDE: Tampering. See §8 T-006 for the full register row.

Attack Steps

1. `routes/userProfile.ts:62` calls eval(code) where code is derived from the username field: const code = .
2. username .
3. When an authenticated user sets their username to a JavaScript expression (e.g.

Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Attacker
    participant App
    Note over App: backend-api - routes/userProfile.ts:62
    Attacker->>App: Crafted request exercising the weakness at routes/userProfile.ts line 62
    App->>App: Vulnerable branch executes without the missing control
    App-->>Attacker: Response confirms the weakness - CWE-95
```

Defense in Depth

- Primary mitigation: M-006 (Remove `eval()` from username processing; use safe string rendering)

3.7 DOM XSS search results rendered via trust HTML bypass

Source: F-019 — DOM XSS search results rendered via trust HTML bypass - `frontend/src/app/search-result/search-result.component.ts:170`

Severity **Critical** (CWE-79). STRIDE: Tampering. See §8 T-019 for the full register row.

Attack Steps

1. `frontend/src/app/search-result/search-result.component.ts:170` calls this.sanitizer.bypassSecurityTrustHtml(queryParam) and binds the result to [innerHTML] in the template.
2. An attacker crafts a URL with q=`<img src=x onerror=alert(document.cookie)>` and shares it.
3. When the victim clicks the link, the Angular SPA renders the unsanitized parameter, executing the attacker's JavaScript in the victim's browser with full access to localStorage (JWT token).

Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Attacker
    actor Victim
    participant App
    Note over App: frontend-spa - frontend/src/app/search-result/search-result.component.ts:170
    Attacker->>App: Reflected attacker-controlled input (HTML payload) with HTML payload - onerror
handler exfiltrates cookies
    Victim->>App: Loads page rendering the stored / reflected payload
    App-->>Victim: HTML contains the unescaped attacker payload - script executes
    Victim->>Attacker: Outbound request leaks session cookie / token
```

Defense in Depth

- Primary mitigation: M-019 (Remove bypassSecurityTrustHtml; use Angular's default sanitization)

3.8 Stored XSS feedback comments rendered unsanitized in admin…

Source: F-020 — Stored XSS feedback comments rendered unsanitized in admin panel - `frontend/src/app/administration/administration.component.ts:78`

Severity **Critical** (CWE-79). STRIDE: Tampering. See §8 T-020 for the full register row.

Attack Steps

1. `frontend/src/app/administration/administration.component.ts:78` marks feedback.comment as trusted HTML via bypassSecurityTrustHtml(feedback.comment).
2. The admin panel at `/administration` renders these comments via [innerHTML] binding.
3. Any user who submits feedback with XSS payload stores it in the database; when admin views the panel, the payload executes in the admin's browser, potentially stealing the admin JWT and enabling privilege escalation.

Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Attacker
    actor Victim
    participant App
    Note over App: frontend-spa - frontend/src/app/administration/administration.component.ts:78
    Attacker->>App: Stored feedback / comment submission (HTML payload) with HTML payload - onerror
handler exfiltrates cookies
    Victim->>App: Loads page rendering the stored / reflected payload
    App-->>Victim: HTML contains the unescaped attacker payload - script executes
    Victim->>Attacker: Outbound request leaks session cookie / token
```

Defense in Depth

- Primary mitigation: M-020 (Sanitize feedback content server-side before storage; remove bypassSecurityTrustHtml in admin panel)

3.9 MD5 password hashing passwords trivially reversible

Source: F-027 — MD5 password hashing passwords trivially reversible - `lib/insecurity.ts:43`

Severity **Critical** (CWE-916). STRIDE: Information Disclosure. See §8 T-027 for the full register row.

Attack Steps

1. `models/user.ts:77` and `lib/insecurity.ts:43` hash passwords with a single-round MD5 with no salt: crypto.createHash('md5').update(data).digest('hex').
2. MD5 is a fast, cryptographically broken hash.
3. The complete md5 rainbow table for common passwords fits in memory.

Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Attacker
    participant App
    Note over App: backend-api - lib/insecurity.ts:43
    Attacker->>Attacker: Craft JWT with header alg=none and admin claims
    Attacker->>App: Crafted HTTP request to the affected endpoint with the unsigned token in
Authorization
    App->>App: Validate token without pinning the signing algorithm
    App-->>Attacker: 200 OK - request authorised under forged admin identity
```

Defense in Depth

- Primary mitigation: M-027 (Replace MD5 with bcrypt (cost factor 12+) for password hashing)

---

4. Assets

Information assets and the classification level that drives the Confidentiality / Integrity / Availability targets used in §8 Threat Register risk scoring.

Asset	ID	Classification	Description	Linked Threats
User Credentials (email + MD5 password)	A-001	Confidential	User email addresses and MD5-hashed passwords stored in the SQLite Users table. MD5 with no salt makes them trivially reversible.	F-003 (SQL injection login authentication bypass) · F-004 (SQL injection product search) · F-016 (Missing rate limiting on authentication endpoint) · F-019 (DOM XSS search results rendered via trust HTML bypass) · F-020 (Stored XSS feedback comments rendered unsanitized in admin panel) · F-021 (XSS last login ip rendered as trusted HTML) · F-024 (XSS via postMessage and iframe DOM based attack) · F-025 (Product description and review rendered as HTML stored XSS vector) · F-027 (MD5 password hashing passwords trivially reversible)
RSA Private Key (JWT signing)	A-002	Restricted	2048-bit RSA private key hardcoded in `lib/insecurity.ts` line 23. Used to sign all JWTs. Permanently compromised as it is committed to the public GitHub repository.	F-001 (JWT algorithm confusion algorithm none bypass) · F-002 (RSA private key hardcoded offline JWT forgery) · F-008 (Zip slip path traversal via unzipper 0.9.15) · F-010 (Sensitive files and logs exposed without authentication) · F-013 (Prometheus metrics endpoint exposed without authentication) · F-015 (Path traversal in data erasure layout parameter) · F-028 (User model returns sensitive fields in API responses) · F-030 (Database schema exposure via SQL injection in search route)
JWT Session Tokens	A-003	Confidential	JWT tokens stored in browser localStorage. Signed with RS256 but the private key is public. Tokens include user id, email, role (isAdmin), and basket id.	F-002 (RSA private key hardcoded offline JWT forgery) · F-008 (Zip slip path traversal via unzipper 0.9.15) · F-010 (Sensitive files and logs exposed without authentication) · F-013 (Prometheus metrics endpoint exposed without authentication) · F-015 (Path traversal in data erasure layout parameter) · F-022 (Client side JWT stored in localStorage accessible to XSS) · F-028 (User model returns sensitive fields in API responses) ·

Asset	ID	Classification	Description	Linked Threats
				F-030 (Database schema exposure via SQL injection in search route)
SQLite Database (All Application Data)	A-004	Confidential	SQLite3 database containing all users, products, orders, complaints, feedback, payment cards, addresses, and other relational data. Accessible via raw SQL injection in login and search routes.	F-003 (SQL injection login authentication bypass) · F-004 (SQL injection product search) · F-011 (IDOR basket accessible by any authenticated user) · F-019 (DOM XSS search results rendered via trust HTML bypass) · F-020 (Stored XSS feedback comments rendered unsanitized in admin panel) · F-021 (XSS last login ip rendered as trusted HTML) · F-024 (XSS via postMessage and iframe DOM based attack) · F-025 (Product description and review rendered as HTML stored XSS vector) · F-029 (Product reviews allow unauthorized update IDOR in review update) · F-030 (Database schema exposure via SQL injection in search route)
MarsDB Document Store (Reviews + Orders)	A-005	Internal	In-memory MongoDB-compatible store containing product reviews and orders. Vulnerable to NoSQL injection via $where operator in `showProductReviews.ts`.	-
Application Access Logs	A-006	Internal	Morgan combined-format access logs written to logs/ directory. Intentionally exposed at `/support/logs` without authentication - readable by anyone.	F-010 (Sensitive files and logs exposed without authentication) · F-013 (Prometheus metrics endpoint exposed without authentication) · F-014 (Access logs exposed to unauthenticated users tamper potential) · F-028 (User model returns sensitive fields in API responses) · F-030 (Database schema exposure via SQL injection in search route)
Uploaded Files (Complaints, Profile Images)	A-007	Internal	Files uploaded via `/file-upload` and `/profile/image/file` stored in uploads/complaints/ and assets/public/images/uploads/. Zip-slip allows	F-005 (Remote code execution notevil sandbox bypass in B2B API) · F-006 (Server side code injection via eval on username field) · F-007 (XXE via XML file upload with external entity resolution) · F-008 (Zip slip path traversal via unzipper

Asset	ID	Classification	Description	Linked Threats
			extraction to arbitrary paths.	0.9.15) · F-009 (Server side request forgery via profile image URL upload) · F-015 (Path traversal in data erasure layout parameter)
Server-Side Application Code and Config	A-008	Internal	TypeScript source, compiled JS, config YAML files (config/*.yml), and intentional vulnerability snippets accessible via /snippets/ routes.	-
Prometheus Metrics Endpoint	A-009	Internal	prom-client metrics exposed at `/metrics` without authentication. Reveals server performance data, request counts, and system information.	-
Encryption Keys Directory	A-010	Restricted	The /encryptionkeys/ route serves the RSA public key (`jwt.pub`) and other key material with directory listing enabled. The private key is embedded in source.	F-002 (RSA private key hardcoded offline JWT forgery) · F-008 (Zip slip path traversal via unzipper 0.9.15) · F-010 (Sensitive files and logs exposed without authentication) · F-013 (Prometheus metrics endpoint exposed without authentication) · F-015 (Path traversal in data erasure layout parameter) · F-028 (User model returns sensitive fields in API responses) · F-030 (Database schema exposure via SQL injection in search route)

5. Attack Surface

Network-reachable entry points classified by authentication requirement. Each row links to the threat(s) referenced in its **Notes** column. The **Risk** column reflects the highest-severity linked finding.

5.1 Unauthenticated Entry Points (11)

Method	Route	Auth	Risk	Notes
POST	`/b2b/v2/orders`	No	Critical	F-005 (Remote code execution notevil sandbox bypass in B2B API) RCE via notevil sandbox escape. User-supplied JavaScript executed in vm context.
GET	`/rest/products/search`	No	Critical	F-004 (SQL injection product search) F-030 (Database schema exposure via SQL injection in search route) SQL injection (`search.ts:23`) - raw string interpolation in LIKE clause.
POST	`/rest/user/login`	No	Critical	F-016 (Missing rate limiting on authentication endpoint) F-003 (SQL injection login authentication bypass) SQL injection (`login.ts:34`) - raw string interpolation in WHERE clause. No rate limiting beyond `/rest/user/reset-password` .
GET	`/encryptionkeys/`	No	High	F-010 (Sensitive files and logs exposed without authentication) Serves RSA public key and directory listing of encryption key files.
POST	`/file-upload`	No	High	F-007 (XXE via XML file upload with external entity resolution) F-008 (Zip slip path traversal via unzipper 0.9.15) F-009 (Server side request forgery via profile image URL upload) XXE (libxmljs2 `noent:true`), zip-slip (unzipper 0.9.15), YAML deserialization.
GET	`/ftp/`	No	High	F-010 (Sensitive files and logs exposed without authentication) Directory listing of FTP files. serve-index middleware exposed without auth.
GET	`/support/logs/`	No	High	F-010 (Sensitive files and logs exposed without authentication) F-014 (Access logs exposed to unauthenticated users tamper potential) Serves application log files with directory listing. No authentication.
GET	`/metrics`	No	Medium	F-013 (Prometheus metrics endpoint exposed without authentication) Prometheus metrics endpoint - exposes server internals without authentication.

Method	Route	Auth	Risk	Notes
GET	`/redirect`	No	Medium	F-012 (Open redirect bypass via allowlist prefix check) Open redirect via `isRedirectAllowed()` bypass - redirects to attacker-controlled URL.
GET	`/api-docs`	No	-	Swagger UI with full API documentation exposed publicly.
?	`Socket.IO WebSocket connection`	No	-	`Socket.IO` 3.x real-time channel. Unauthenticated connections accepted on initial connect.

5.2 Authenticated Entry Points (3)

Method	Route	Auth	Risk	Notes
POST	`/profile/image/url`	Yes	Critical	F-009 (Server side request forgery via profile image URL upload) F-006 (Server side code injection via eval on username field) SSRF - user-controlled URL fetched server-side via `fetch()`. No allowlist validation.
GET	`/rest/basket/:id`	Yes	High	F-011 (IDOR basket accessible by any authenticated user) IDOR - any authenticated user can access any basket by guessing the numeric id.
POST	`/rest/products/:id/reviews (PUT variant)`	Yes	High	F-029 (Product reviews allow unauthorized update IDOR in review update) F-026 (NoSQL injection via $where operator in MarsDB reviews) F-031 (JavaScript injection via $where can crash MarsDB process) NoSQL injection via $where in MarsDB query (`showProductReviews.ts:36`).

---

7. Security Architecture

This chapter is organized by security-control category. The architecture section avoids artificial control IDs and finding-ID columns in overview tables. Findings are listed only where the affected control is described.

§7 schema v2 (13-section control-category layout). Cataloged controls: 27 total - 1 adequate, 3 partial, 13 weak, 0 unsafe, 10 missing. Linked threats: 31.

How to read the verdicts. Every control category (and every sub-control below it) carries exactly one status. The two red verdicts do **not** mean the same thing - this is the distinction that decides what you have to do about a finding:

Status	Meaning	What it asks of you
✅ Adequate	Control is present and sound	Nothing - keep it
🟡 Partial	Present, but with meaningful gaps	Close the gap
🟠 Weak	Present, but has exploitable gaps	Strengthen it
🔴 Unsafe	**Present and relied upon, but defeated / trivially bypassable**	**Fix the existing control**
⚫ Missing	**Control was never built**	**Add the control**
-	Not applicable to this codebase	-

So "🔴 Unsafe" on a control category does not mean the control is absent - it means the control exists but does not hold (e.g. an MD5 password hash, a raw-SQL query path, a hardcoded signing key). "⚫ Missing" is reserved for controls that were never built (e.g. no Content-Security-Policy header).

7.1 Security Control Overview

Control category	Verdict	Main reason
7.2 Identity and Authentication Controls	Weak	3 routed findings; catalogued controls are weak (e.g. JWT Authentication (express-jwt + jsonwebtoken), Password-Based Authentication).
7.3 Session and Token Controls	Missing	1 routed finding; no controls catalogued for this category.
7.4 Authorization Controls	Missing	4 routed findings; no controls catalogued for this category.
7.5 Query Construction and Data Access Controls	Weak	3 routed findings; catalogued controls are weak (e.g. SQL Query Parameterization, NoSQL Query Parameterization).
7.6 Input Boundary Validation Controls	Weak	1 routed finding; no compensating controls catalogued.
7.7 Output Encoding and Rendering Controls	Weak	5 routed findings; catalogued controls are weak (e.g. XSS Output Encoding (Angular DomSanitizer), Server-Side Template Rendering).
7.8 Browser and Cross-Origin Controls	Missing	No controls catalogued for this category.
7.9 Cryptography Secrets and Data Protection	Missing	1 routed finding; no controls catalogued for this category.
7.10 File Parser and Outbound Request Controls	Missing	11 routed findings; no controls catalogued for this category.
7.11 Operations Runtime and Supply Chain Controls	Missing	2 routed findings; no controls catalogued for this category.
7.12 Real-time and Not Applicable Controls	-	No controls or findings routed to this category.
7.13 Defense-in-Depth Summary	-	No controls or findings routed to this category.

7.2 Identity and Authentication Controls

Verdict: Weak - password hashing is a single-round unsalted MD5; TOTP is optional and unenforced for admin accounts; the password login query is raw SQL susceptible to injection bypass.

Controls covered: 7.2.1 Password-Based Authentication · 7.2.2 Two-Factor Authentication (TOTP)

- 7.2.1 Password-Based Authentication
- 7.2.2 Two-Factor Authentication (TOTP)

Implemented controls: `routes/login.ts:23` (password credential check), `lib/insecurity.ts:43` (password hashing), `routes/2fa.ts` (TOTP enrollment and verification).

Assessment: Authentication is implemented through two user-facing mechanisms: password-based login and optional TOTP as a second factor. Each has a structural gap. The password login route is vulnerable to SQL injection bypass and stores passwords as unsalted MD5 hashes. TOTP is correctly implemented but opt-in with no enforcement for high-privilege accounts. The JWT token issued on successful authentication - including its signing, algorithm enforcement, and key management - is described in §7.3 Session and Token Controls.

7.2.1 Password-Based Authentication

Status: ⚠ Weak - login query is raw SQL susceptible to injection bypass; passwords are stored as unsalted MD5 hashes.

`routes/login.ts:34` handles credential verification by constructing a raw SQL query that selects the user record matching the supplied email and pre-hashed password. `lib/insecurity.ts:43` provides the hashing helper: a single call to `crypto.createHash('md5')` with no salt. Both the query construction and the hash function are exploitable independently.

The diagram shows the intended password login path:

```
sequenceDiagram
    autonumber
    actor User
    participant SPA as Login Form
    participant API as routes/login.ts
    participant DB as SQLite Users table

    User->>SPA: POST email + password
    SPA->>API: POST /rest/user/login
    API->>API: hash(password) via lib/insecurity.ts:43
    API->>DB: SELECT user WHERE email=? AND password=?
    DB-->>API: user row
    API->>API: issue JWT via lib/insecurity.ts:issueAuthToken
    API-->>SPA: { token, basket_id, email }
```

Security assessment

Two independent weaknesses sit on the login path:

- `routes/login.ts:34` concatenates `req.body.email` directly into `models.sequelize.query()`. The payload `' OR 1=1--` short-circuits the WHERE clause and returns the seeded admin row without a password (F-003 — SQL injection login authentication bypass).
- `lib/insecurity.ts:43` hashes passwords with unsalted MD5. Any database dump - obtained through the SQL injection above or otherwise - yields plaintext for every account via a rainbow table lookup (F-027 — MD5 password hashing passwords trivially reversible).

The raw SQL construction at the heart of the login route is:

```
models.sequelize.query(
  `SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${req.body.password ||
       ''}' AND deletedAt IS NULL`,
  { model: UserModel, plain: true }
)
```

Relevant findings

- F-003 — SQL injection in the login query allows authentication bypass without a valid credential.
- F-027 — Unsalted MD5 hashing means any acquired password hash is trivially reversible offline.

7.2.2 Two-Factor Authentication (TOTP)

Status: ◻ Partial - TOTP enrollment and verification work correctly, but the second factor is entirely optional and never required for admin accounts.

TOTP is implemented via `otplib` in `routes/2fa.ts`. Users may set up a time-based one-time password authenticator through a QR-code enrollment flow. When TOTP is configured on an account, the `/rest/user/login` route detects the presence of a `totpSecret` field and requires a valid OTP before issuing the JWT. Enrollment and verification are correctly implemented.

The diagram shows the TOTP-enabled login path:

```
sequenceDiagram
    autonumber
    actor User
    participant SPA as Login Form
    participant API as routes/login.ts + routes/2fa.ts
    participant OTP as otplib verifier

    User->>SPA: POST email + password
    SPA->>API: POST /rest/user/login
    API->>API: verify password (see §7.2.1)
    API->>API: check totpSecret present on account
    API-->>SPA: 401 with totp challenge token
    SPA->>User: prompt for TOTP code
    User->>SPA: enter 6-digit OTP
    SPA->>API: POST /rest/2fa/verify {token, totp}
    API->>OTP: otplib.authenticator.check(totp, secret)
    OTP-->>API: valid
    API-->>SPA: JWT issued
```

Security assessment

TOTP enrollment and verification are correctly implemented using `otplib`. The control weakness is governance, not code: no route or middleware enforces TOTP enrollment for users with elevated privileges. An attacker who forges an admin JWT (via F-001 — JWT

algorithm confusion algorithm none bypass or F-002 — RSA private key hardcoded offline JWT forgery) bypasses TOTP entirely because the 2FA check only runs on the password login path, not on JWT-protected routes.

Relevant findings

- F-016 — No rate-limiting on the login endpoint allows credential brute-force that also exhausts the TOTP window.

7.3 Session and Token Controls

Verdict: ✗ Missing - JWT is stored in browser `localStorage`, exposed to XSS theft; no revocation mechanism exists; session invalidation on logout is absent; the JWT library accepts unsigned tokens.

Controls covered: 7.3.1 JWT Bearer Token Validation (express-jwt middleware) · 7.3.2 Session Token Storage (Browser localStorage) · 7.3.3 Session Invalidation and Logout

- 7.3.1 JWT Bearer Token Validation (express-jwt middleware)
- 7.3.2 Session Token Storage (Browser localStorage)
- 7.3.3 Session Invalidation and Logout

Implemented controls: `lib/insecurity.ts:issueAuthToken()` issues RS256 JWTs on login; `lib/insecurity.ts:54-56` (JWT issuance and validation via `express-jwt@0.1.3`); `routes/login.ts` issues and `server.ts:289` configures a session cookie with the hardcoded `'kekse'` secret.

Assessment: This application uses a single locally-signed token format (JWT) for every authenticated session. The sub-sections below trace one token through its lifecycle: validation on protected routes (the JWT middleware), storage in the browser, and the absence of revocation and session-invalidation controls.

7.3.1 JWT Bearer Token Validation (express-jwt middleware)

Status: ✗ Weak - `express-jwt@0.1.3` accepts `alg:none` tokens; algorithm enforcement is absent and the library predates the CVE-2015-9235 fix.

JWT validation is performed by `express-jwt@0.1.3` mounted in `lib/insecurity.ts:54-56`. On every protected route the middleware calls `jsonwebtoken@0.4.0` to decode the `Authorization: Bearer` header and attach the payload to `req.user`. The library versions in use predate the security fixes introduced in `jsonwebtoken` 6.x - neither enforces the signing algorithm.

The diagram shows the intended positive path for a protected request carrying a valid JWT:

```
sequenceDiagram
    autonumber
    actor User
    participant API as Express Route
    participant MW as express-jwt middleware
    participant Key as RSA Public Key

    User->>API: GET /rest/user/whoami (Bearer token)
    API->>MW: verify(token)
    MW->>Key: load jwt.pub
    Key-->>MW: PEM bytes
    MW-->>API: decoded payload {id, email, role}
    API-->>User: 200 OK - user data
```

Security assessment

Two independent weaknesses break the JWT verification boundary:

- `express-jwt@0.1.3` does not enforce the `algorithms` option. Supplying `{"alg":"none"}` in the JWT header produces a token with an empty signature that the library accepts as valid (F-001 — JWT algorithm confusion algorithm none bypass).
- The RS256 private key is hardcoded at `lib/insecurity.ts:23`. Anyone with repo read access can call `jwt.sign(payload, privateKey, {algorithm:'RS256'})` to forge arbitrary tokens, bypassing signature verification entirely (F-002 — RSA private key hardcoded offline JWT forgery).

Relevant findings

- F-001 — `alg:none` acceptance lets an attacker craft a completely unsigned token and set `role=admin` without knowing the private key.
- F-002 — Hardcoded RSA private key at `lib/insecurity.ts:23` enables offline forgery of fully-signed RS256 tokens.

7.3.2 Session Token Storage (Browser localStorage)

Status: □ Missing - JWT is stored in `localStorage`, which is readable by any XSS payload; `HttpOnly` cookies are not used.

On successful login, `routes/login.ts` returns the JWT in the JSON response body. The Angular frontend stores it in `localStorage` via `TokenService` in `frontend/src/app/Services/token.service.ts`. The session cookie configured in `server.ts:289` uses the hardcoded secret `'kekse'` but is separate from the JWT mechanism; the JWT itself is never placed in an `HttpOnly` cookie.

Security assessment

`localStorage` is accessible to any JavaScript running in the page origin. The stored XSS vectors in F-019 — DOM XSS search results rendered via trust HTML bypass, F-020 — Stored XSS feedback comments rendered unsanitized in admin panel, and F-021 — XSS last login ip

rendered as trusted HTML can read `localStorage` directly, allowing session hijacking with a single `document.cookie`-style payload. Moving the JWT to an `HttpOnly` `SameSite=Strict` cookie would remove this access entirely.

Relevant findings

- F-022 — JWT in `localStorage` is readable by every XSS payload in this codebase, enabling full session takeover.

7.3.3 Session Invalidation and Logout

Status: □ Weak - logout clears the client-side `localStorage` entry but does not invalidate the JWT server-side; stolen tokens remain valid until natural expiry.

`routes/logout.ts` responds to `GET /rest/user/logout`. The server does not maintain a token allowlist or denylist; the JWT's server-side validity depends solely on its signature and expiry claim. Client-side logout removes the `localStorage` entry, which means a token stolen before logout continues to be accepted by every `express-jwt`-protected route.

Security assessment

Server-side token revocation is absent. A JWT stolen via any of the XSS vectors (F-019 — DOM XSS search results rendered via trust HTML bypass, F-020 — Stored XSS feedback comments rendered unsanitized in admin panel) continues to authenticate requests after the legitimate user has logged out. A revocation list keyed on the JWT `jti` claim, or a short expiry combined with refresh-token rotation, would close this gap.

Relevant findings

- F-022 — Stolen JWT remains valid server-side after client logout, enabling persistent impersonation.

7.4 Authorization Controls

Verdict: □ Missing - object-level authorization checks are absent on basket, wallet, and address routes; role checks rely on a client-side JWT claim that is forgeable.

Controls covered: 7.4.1 Role-Based Access Control (isAdmin JWT claim) · 7.4.2 Object-Level Authorization

- 7.4.1 Role-Based Access Control (isAdmin JWT claim)
- 7.4.2 Object-Level Authorization

Implemented controls: `lib/insecurity.ts:54-55` (JWT parsing middleware that attaches `req.user`), `server.ts` route middleware (presence check only).

Assessment: Authorization has two layers, both broken. Role checks read `req.user.role` from the JWT payload - a value that is forgeable via the hardcoded key (F-002 — RSA private key hardcoded offline JWT forgery) or `alg:none` bypass (F-001 — JWT algorithm confusion

algorithm none bypass). Object-level checks on resource-owning endpoints accept `req.body.UserId` rather than the authenticated user's id from the verified JWT, so any logged-in user can operate on another user's resources by supplying a different id.

7.4.1 Role-Based Access Control (isAdmin JWT claim)

Status: ⚠ Weak - role check reads `req.user.role` from the JWT payload, which is forged via the hardcoded signing key; Angular route guards enforce nothing server-side.

Admin routes are guarded by middleware in `server.ts` that checks `req.user.role === 'admin'` after the `express-jwt` middleware attaches the decoded payload. The admin panel UI is additionally gated by Angular route guards in `frontend/src/app/app.routing.module.ts`. Both checks rely on the JWT claim being trustworthy.

Security assessment

Because the RSA private key is committed to the repository (F-002 — RSA private key hardcoded offline JWT forgery) and `alg:none` is accepted (F-001 — JWT algorithm confusion algorithm none bypass), any attacker can mint a JWT with `role=admin`. The server-side role check then passes transparently. Angular route guards (F-023 — Angular route guards bypassable client side) add no protection because they run only in the browser and are bypassed by making direct API calls.

Relevant findings

- F-023 — Angular route guards are client-only; direct API calls to admin endpoints are not blocked server-side.
- F-017 — Mass assignment on wallet routes accepts `req.body.UserId`, bypassing role-based ownership.

7.4.2 Object-Level Authorization

Status: ⚠ Missing - basket, wallet, and address routes use the user id from `req.body` rather than from the verified JWT; cross-user access requires only a valid session token.

`routes/basket.ts`, `routes/wallet.ts`, and `routes/address.ts` accept a `UserId` parameter from the request body and use it directly to query the database. No comparison is made between the supplied id and the authenticated user's id from `req.user.id`.

Security assessment

Any authenticated user can read or modify any other user's basket, wallet balance, or address by changing the `UserId` field in the request body. This is a horizontal privilege escalation: authentication is required, but the authorization check that would restrict a user to their own resources is absent.

The wallet update route illustrates the pattern:

```
// routes/wallet.ts — UserId from body, not from verified JWT
const user = await UserModel.findOne({ where: { id: req.body.UserId } })
```

Relevant findings

- F-011 — IDOR on basket route: any authenticated user can retrieve another user's basket contents.
- F-017 — Mass assignment on wallet: `req.body.UserId` determines whose balance is updated.

7.5 Query Construction and Data Access Controls

Verdict: ❌ Weak - SQL and NoSQL injection are present on unauthenticated routes; Sequelize parameterized queries are bypassed by two raw `models.sequelize.query()` call sites.

Controls covered: 7.5.1 SQL Query Parameterization (Sequelize + Raw Queries) · 7.5.2 NoSQL Query Parameterization (MarsDB)

- 7.5.1 SQL Query Parameterization (Sequelize + Raw Queries)
- 7.5.2 NoSQL Query Parameterization (MarsDB)

Implemented controls: Sequelize ORM backs most model operations; `routes/login.ts:34` and `routes/search.ts:23` call `models.sequelize.query()` directly; `routes/showProductReviews.ts:36` passes user input to a MarsDB `$where` clause.

Assessment: Sequelize's parameterized query support is available and used by most routes. Two routes opt out and construct raw SQL strings; one additional route passes user-controlled structure to a MarsDB selector. All three deviations are on high-traffic paths - login and product search are unauthenticated, and review queries are authenticated but trivially reachable. Fixing the login and search routes removes two Critical findings and eliminates F-030 — Database schema exposure via SQL injection in search route schema exposure as a side-effect.

7.5.1 SQL Query Parameterization (Sequelize + Raw Queries)

Status: ❌ Weak - login and search routes build SQL strings by concatenating user input rather than using Sequelize bind parameters.

Sequelize models back most relational data access in this codebase. Two routes deviate: `routes/login.ts:34` and `routes/search.ts:23` both construct raw SQL strings via `models.sequelize.query()`. The login route interpolates `req.body.email` and `req.body.password` directly; the search route interpolates the `q` query parameter into a `LIKE` clause.

The product search route illustrates the raw SQL construction that bypasses parameter binding:

```
models.sequelize.query(
  `SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') AND
        deletedAt IS NULL) ORDER BY name`
)
```

Security assessment

Both raw-SQL sites are on unauthenticated endpoints, making exploitation trivial:

- `routes/login.ts:34` - `' OR 1=1--` in the email field returns the seeded admin account row and bypasses the password check (F-003 — SQL injection login authentication bypass).
- `routes/search.ts:23` - a UNION-based payload dumps the full Users table including password hashes and the SQLite schema (F-004 — SQL injection product search (UNION + schema exfil), F-030 — Database schema exposure via SQL injection in search route).

Relevant findings

- F-003 — SQL injection in the login query allows authentication bypass without a valid credential.
- F-004 — SQL injection in the search route enables UNION-based data exfiltration and schema disclosure.
- F-030 — Database schema exposed as a side-effect of the search injection; eliminated when F-004 (SQL injection product search) is fixed.

7.5.2 NoSQL Query Parameterization (MarsDB)

Status: Weak - review queries accept a `$where` JavaScript expression from user-supplied input, giving the attacker control over query logic.

Product reviews are stored in MarsDB, a MongoDB-compatible in-memory database. `routes/showProductReviews.ts:36` passes the user-supplied `id` parameter into a MarsDB selector. MarsDB supports the `$where` operator, which evaluates a JavaScript string as a boolean predicate over each document.

Security assessment

Supplying `{"$where":"sleep(2000)"}` or similar payloads as the review `id` executes JavaScript inside the MarsDB process. This enables both blind data extraction and a denial-of-service via CPU-intensive expressions that crash the MarsDB worker (F-026 (NoSQL injection via $where operator in MarsDB reviews), F-031 (JavaScript injection via $where can crash MarsDB process)).

Relevant findings

- F-026 — NoSQL injection via `$where` allows JavaScript execution in the MarsDB query context.
- F-031 — JavaScript injection in `$where` can crash the MarsDB process through resource exhaustion.

7.6 Input Boundary Validation Controls

Verdict: □ Weak - upload file-type and path-traversal constraints are absent; `multer` enforces only a file-size limit.

Controls covered: 7.6.1 Input Boundary Validation (multer + path checks)

- 7.6.1 Input Boundary Validation (multer + path checks)

Implemented controls: `multer` enforces a maximum file size on upload endpoints; `routes/dataErasure.ts:69` calls `path.resolve()` on the layout parameter without subsequent containment check.

Assessment: Input validation is limited to file size. File type, content-type, and path-traversal constraints are absent on the upload endpoint and the layout parameter route. Uploaded XML, ZIP, and YAML files reach parsers with dangerous defaults enabled without any pre-flight type check.

7.6.1 Input Boundary Validation (multer + path checks)

Status: □ Weak - file-size limit is enforced; file-type allow-listing and path-containment checks are absent.

`multer` is configured on the file upload endpoint to reject files above a size threshold. No MIME-type or extension allow-list is applied before passing uploaded files to `libxmljs2` (XML), `unzipper` (ZIP), or the YAML parser. `routes/dataErasure.ts:69` calls `path.resolve(req.body.layout)` without subsequently verifying that the resolved path stays within the permitted directory.

Security assessment

Three upload-validation gaps compound each other:

- No file-type check allows `.xml` uploads to reach `libxmljs2` with `noent:true` (F-007 — XXE via XML file upload with external entity resolution).
- No archive extraction path check allows ZIP files with `../` entries to write outside the target directory (F-008 — Zip slip path traversal via unzipper 0.9.15).
- `routes/dataErasure.ts:69` resolves the `layout` parameter without a containment check, allowing traversal to read arbitrary files (F-015 — Path traversal in data erasure layout parameter).

Relevant findings

- F-031 — JavaScript injection in MarsDB via unvalidated input structure crashes the process.

7.7 Output Encoding and Rendering Controls

Verdict: ⚠ Weak - Angular's default output encoding is disabled via `bypassSecurityTrustHtml()` in 14+ components; server-side template rendering relies on user input in `eval()`.

Controls covered: 7.7.1 XSS Output Encoding (Angular DomSanitizer) · 7.7.2 Server-Side Template Rendering (eval-based)

- 7.7.1 XSS Output Encoding (Angular DomSanitizer)
- 7.7.2 Server-Side Template Rendering (eval-based)

Implemented controls: Angular's template engine escapes by default; `DomSanitizer.bypassSecurityTrustHtml()` is called explicitly to override this in `search-result.component.ts:170` and 13+ other components.

Assessment: Angular's default output encoding would prevent all DOM-XSS findings in this codebase. Every XSS finding here exists because `bypassSecurityTrustHtml()` was called deliberately to render HTML from user-controlled sources. Removing these calls and letting Angular's default sanitizer run is the single fix that closes five findings simultaneously.

7.7.1 XSS Output Encoding (Angular DomSanitizer)

Status: ⚠ Weak - Angular's default escaping is present but overridden at 14+ call sites via `bypassSecurityTrustHtml()`; every override is a live XSS sink.

Angular templates escape interpolated values by default. Throughout the frontend, `DomSanitizer.bypassSecurityTrustHtml()` is called before binding user-controlled strings to `[innerHTML]` properties. The primary sites are `search-result.component.ts:170` (search query reflected in results), `administration.component.ts` (user feedback rendered in the admin panel), and `last-login-ip.component.ts` (login IP address rendered as HTML).

Security assessment

Each `bypassSecurityTrustHtml()` call converts a safe Angular template binding into a raw HTML injection sink. Three exploitable patterns:

- `search-result.component.ts:170` - the search query parameter is reflected as trusted HTML, enabling DOM XSS (F-019 — DOM XSS search results rendered via trust HTML bypass).
- `administration.component.ts` - stored feedback content from any user is rendered unsanitized in the admin panel (F-020 — Stored XSS feedback comments rendered unsanitized in admin panel).
- `last-login-ip.component.ts` - the last-login IP, controllable via headers, is rendered as trusted HTML (F-021 — XSS last login ip rendered as trusted HTML).

Relevant findings

- F-019 — DOM XSS via search query reflected through `bypassSecurityTrustHtml()`.

- F-020 — Stored XSS in admin panel feedback view via unsanitized `bypassSecurityTrustHtml()` call.
- F-021 — XSS via last-login IP field rendered as trusted HTML.
- F-025 — Product descriptions stored with HTML and rendered without sanitization in product detail views.

7.7.2 Server-Side Template Rendering (eval-based)

Status: □ Weak - `routes/userProfile.ts:62` passes the username field through `eval()` for Handlebars template processing, enabling server-side code injection.

`routes/userProfile.ts:62` calls `eval()` to process the username field as a Handlebars template. The intent is to allow personalised template expressions in user profiles. The result is that any JavaScript expression in the username field executes in the server process context.

Security assessment

`eval()` on user-controlled input is a direct code injection sink, not a template rendering issue. Any authenticated user can set their username to `{{require('child_process').execSync('id')}}` and retrieve server-side process output. This is a Critical finding independent of the template intent; see §7.10 for the related B2B eval sink.

Relevant findings

- F-006 — Server-side code injection via `eval()` on the username field grants arbitrary RCE to any authenticated user.

7.8 Browser and Cross-Origin Controls

Verdict: □ Missing - no Content-Security-Policy at app level; CORS is configured as a wildcard; CSRF protection is absent.

Controls covered: 7.8.1 Content Security Policy (Helmet partial) · 7.8.2 CORS Policy (wildcard) · 7.8.3 CSRF Protection

- 7.8.1 Content Security Policy (Helmet partial)
- 7.8.2 CORS Policy (wildcard)
- 7.8.3 CSRF Protection

Implemented controls: `helmet@4.6` is mounted in `server.ts:187` with a non-default configuration; `cors@2.8.5` is mounted in `server.ts:181-182`; no CSRF token middleware is present.

Assessment: The three browser-control primitives that would contain XSS and CSRF impact are all either absent or misconfigured. Helmet's `xssFilter` is explicitly disabled; no app-level CSP header is emitted; CORS allows every origin; and no CSRF token middleware protects state-changing requests. These gaps do not introduce findings on their own but amplify the XSS findings in §7.7 by removing the last browser-side mitigations.

7.8.1 Content Security Policy (Helmet partial)

Status: ⚠ Weak - a per-route CSP is present for the user profile endpoint only; no app-level CSP is emitted; Helmet's `xssFilter` is explicitly disabled.

`server.ts:187` mounts Helmet without a `contentSecurityPolicy` configuration, so no `Content-Security-Policy` header is emitted on most routes. `routes/userProfile.ts:95` adds a route-specific CSP header, but this covers only the profile page. Helmet's `xssFilter` (legacy `X-XSS-Protection: 1; mode=block`) is explicitly disabled in the Helmet options.

Security assessment

Without an app-level CSP, scripts from any origin can be injected through the XSS sinks in §7.7. A CSP `default-src 'self'` with a nonce-based `script-src` would prevent inline-script and cross-origin-script execution even when `bypassSecurityTrustHtml()` is called. The per-route CSP on the profile endpoint does not help because it applies after the `eval()`-based code injection in `routes/userProfile.ts:62`.

Relevant findings

- No dedicated finding routed in this assessment.

7.8.2 CORS Policy (wildcard)

Status: ⚠ Weak - `app.use(cors())` in `server.ts:181` enables wildcard origin and credentials, allowing any origin to make credentialed cross-origin requests.

`server.ts:181-182` mounts the `cors` middleware with default options, which responds to every `Origin` header with `Access-Control-Allow-Origin: *`. No allowlist of trusted origins is configured.

Security assessment

A wildcard CORS policy means any web page on any domain can make cross-origin requests to the API. Combined with the absent CSRF protection (§7.8.3), a malicious page can perform state-changing operations on behalf of a logged-in user. The practical impact is amplified by JWT storage in `localStorage` (which cannot be sent with `SameSite` cookie restrictions).

Relevant findings

- No dedicated finding routed in this assessment.

7.8.3 CSRF Protection

Status: ⚠ Missing - no CSRF token middleware is present; state-changing endpoints accept requests from any origin.

No CSRF token middleware (`csurf` or equivalent) is configured in `server.ts`. State-changing POST, PUT, and DELETE routes do not require any origin-bound token. The session cookie uses a hardcoded secret (`'kekse'`) and no `SameSite` attribute.

Security assessment

All state-changing API endpoints are reachable from cross-origin requests. An attacker-controlled page can trigger basket modifications, order placements, and profile changes on behalf of any user who visits it. Adding `SameSite=Strict` to the session cookie and a CSRF token check on all non-GET routes would close this gap.

Relevant findings

- No dedicated finding routed in this assessment.

7.9 Cryptography Secrets and Data Protection

Verdict: ✗ Missing - three secrets are hardcoded in source; password hashing uses unsalted MD5; the signing key material is permanently compromised.

Controls covered: 7.9.1 Cryptographic Algorithm Selection (MD5, Math.random) · 7.9.2 Secrets Management (hardcoded keys)

- 7.9.1 Cryptographic Algorithm Selection (MD5, Math.random)
- 7.9.2 Secrets Management (hardcoded keys)

Implemented controls: `lib/insecurity.ts:43` (password hashing), `lib/insecurity.ts:55` (CAPTCHA secret), `lib/insecurity.ts:152` (HMAC), `lib/insecurity.ts:23` (RSA private key), `server.ts:289` (cookie secret).

Assessment: All cryptographic secrets are hardcoded string literals in `lib/insecurity.ts` and `server.ts`. The RSA private key is additionally published to the `encryptionkeys/` route. Password hashing uses unsalted MD5, which provides no meaningful resistance to offline attacks. The CAPTCHA secret uses `Math.random()`, which is not cryptographically random. These are design-level choices, not accidental misconfigurations - each must be replaced at the root cause.

7.9.1 Cryptographic Algorithm Selection (MD5, Math.random)

Status: ✗ Weak - passwords are hashed with unsalted MD5; CAPTCHA secrets are generated with `Math.random()`; neither provides meaningful security.

`lib/insecurity.ts:43` calls `crypto.createHash('md5').update(password).digest('hex')` for all password storage and comparison. `routes/captcha.ts:14-19` generates CAPTCHA challenges using `Math.random()`. The RSA algorithm choice (RS256) is sound, but the key material is hardcoded (see §7.9.2).

Security assessment

MD5 without a salt is equivalent to no password hashing for practical purposes: rainbow tables built from dictionary and rule-based sets cover most real-world passwords, and GPU-accelerated MD5 cracking achieves billions of hashes per second. `Math.random()` produces a predictable CAPTCHA that provides no bot protection. Neither weakness requires a cryptographic attack - both are defeated by lookup.

Relevant findings

- F-002 — Unsalted MD5 password hash: any dump obtained through SQL injection yields plaintext credentials immediately.

7.9.2 Secrets Management (hardcoded keys)

Status: ❌ Missing - no runtime secret injection; three secrets are hardcoded literals; the RSA private key is publicly served at `/encryptionkeys`.

Three secrets live as string literals in `lib/insecurity.ts`: the 1024-bit RSA private key (line 23), an HMAC secret (line 152), and the `denyAll()` secret (line 55). A fourth secret - the session cookie signing key `'kekse'` - is hardcoded in `server.ts:289`. The `encryptionkeys/` static route serves the RSA public key file from `encryptionkeys/jwt.pub`, making the key material even more visible.

Security assessment

Cloning the repository gives an attacker both the private key (for JWT forgery) and the cookie secret (for session cookie forgery) with no further access required. Rotating these secrets requires a code change and a deployment - there is no runtime injection path. Moving to environment variables with a secrets manager (or a KMS-backed store for the RSA key) would allow rotation without a code change.

Relevant findings

- F-002 — Hardcoded RSA private key at `lib/insecurity.ts:23`; any repo reader can forge admin JWTs offline.

7.10 File Parser and Outbound Request Controls

Verdict: ❌ Missing - all three upload parser controls (XXE, zip-slip, SSRF) are absent; path-traversal checks are missing on the layout endpoint.

Controls covered: 7.10.1 SSRF Prevention (profile image upload)

- 7.10.1 SSRF Prevention (profile image upload)

Implemented controls: `routes/profileImageUrlUpload.ts:24` fetches a user-supplied URL; `routes/fileUpload.ts` passes uploaded files to `libxmljs2`, `unzipper`, and the YAML parser without pre-flight checks.

Assessment: The file upload endpoint at `POST /file-upload` accepts XML, ZIP, and YAML files and passes them to their respective parsers with dangerous defaults enabled. `libxmljs2` is configured with `noent:true`, which enables external entity resolution. `unzipper@0.9.15` extracts archives without path containment. The profile image URL upload makes an outbound HTTP request to any user-supplied URL without an allowlist. Three independent parser weaknesses on a single endpoint make this the broadest attack surface in the codebase.

7.10.1 SSRF Prevention (profile image upload)

Status: ❌ Missing - `routes/profileImageUrlUpload.ts:24` calls `fetch(req.body.imageUrl)` with no allowlist, SSRF protection library, or redirect-follow limit.

`routes/profileImageUrlUpload.ts:24` accepts a URL in the request body and fetches it with Node's built-in `fetch()`. The fetched content is stored as the user's profile image. No allowlist, blocklist, or SSRF-protection library is applied before the outbound request is made.

Security assessment

Three upload and outbound-request weaknesses share this endpoint area:

- `routes/profileImageUrlUpload.ts:24` - `fetch(req.body.imageUrl)` with no allowlist enables SSRF against internal network hosts (F-009 — Server side request forgery via profile image URL upload).
- `routes/fileUpload.ts:83` - `libxmljs2` parses XML with `noent:true`, resolving external entities to arbitrary URIs including `file:///etc/passwd` (F-007 — XXE via XML file upload with external entity resolution).
- `routes/fileUpload.ts:42-45` - `unzipper@0.9.15` extracts archive entries without verifying that extracted paths stay within the target directory (F-008 — Zip slip path traversal via unzipper 0.9.15).

Relevant findings

- F-009 — SSRF via profile image URL: server fetches any user-supplied URL including internal cloud metadata endpoints.
- F-007 — XXE via XML upload: external entity in uploaded XML resolves to arbitrary file paths.
- F-008 — Zip-slip via archive upload: path traversal entries write outside the intended extraction directory.

7.11 Operations Runtime and Supply Chain Controls

Verdict: ❌ Missing - no SCA scan in CI; critical auth libraries are severely outdated; distroless Docker base image is the only positive supply-chain control.

Controls covered: 7.11.1 Dependency Vulnerability Scanning (CI/SCA) · 7.11.2 Container Security (Dockerfile) · 7.11.3 CI/CD Pipeline Security (GitHub Actions) · 7.11.4 Security Logging and Monitoring (Morgan + Winston) · 7.11.5 Automated SCA scanning

- 7.11.1 Dependency Vulnerability Scanning (CI/SCA)
- 7.11.2 Container Security (Dockerfile)
- 7.11.3 CI/CD Pipeline Security (GitHub Actions)
- 7.11.4 Security Logging and Monitoring (Morgan + Winston)
- 7.11.5 Automated SCA scanning

Implemented controls: `.github/workflows/ci.yml` (no SCA step); `Dockerfile:23-41` (multi-stage distroless image); `.github/workflows/` (16 workflow files, overly broad permissions); `server.ts:338` + `lib/logger.ts` (Morgan + Winston logging).

Assessment: Two critically outdated auth libraries (`jsonwebtoken@0.4.0` , `express-jwt@0.1.3`) and one parser with a known zip-slip vulnerability (`unzipper@0.9.15`) are in production dependencies. No SCA step in CI would catch these. The GitHub Actions workflows use overly broad permissions. On the positive side, the multi-stage Dockerfile produces a distroless final image with no shell or package manager, reducing the container attack surface.

7.11.1 Dependency Vulnerability Scanning (CI/SCA)

Status: ☐ Missing - `.github/workflows/ci.yml` runs no SCA step; `npm audit` or Dependabot are not configured.

The CI pipeline in `.github/workflows/ci.yml` runs tests and lint but contains no `npm audit` , Snyk, or equivalent SCA step. The `package-lock.json` is present and would enable deterministic scanning. The repository is public on GitHub, which enables free Dependabot alerts, but no Dependabot configuration file (`.github/dependabot.yml`) is present.

Security assessment

Without a CI SCA step, three packages with known Critical vulnerabilities (`jsonwebtoken@0.4.0` CVE-2015-9235, `express-jwt@0.1.3` no-algorithm-enforcement, `unzipper@0.9.15` zip-slip) have remained in production dependencies. Adding `npm audit --audit-level=high` as a required CI gate would surface these immediately and block further outdated-dependency regressions.

Relevant findings

- F-014 — Access logs exposed at `/support/logs` ; logging infrastructure is a data exposure vector, not just a monitoring gap.
- F-018 — Error handler exposes internal stack traces in production responses.

7.11.2 Container Security (Dockerfile)

Status: ☐ Partial - distroless final image reduces attack surface; no user privilege drop or seccomp profile is specified.

`Dockerfile:23-41` uses a multi-stage build: a `Node.js` builder stage compiles the TypeScript and Angular frontend, then copies artifacts into a `gcr.io/distroless/nodejs20-debian12` final image. The distroless base has no shell, no package manager, and no OS utilities, which eliminates the most common container post-exploitation paths.

Security assessment

The distroless base is a meaningful positive control. Two gaps remain: the container runs as the default user (root in distroless unless overridden), and no `seccomp` or `AppArmor` profile is specified in the Dockerfile or a default Kubernetes deployment manifest. Neither gap is evidenced in the source, but the absence of a non-root `USER` directive is a standard hardening step that is missing.

Relevant findings

- F-014 — Log files reachable at `/support/logs` are part of the runtime attack surface, not a container issue per se.
- F-018 — Stack-trace exposure in error responses is a runtime configuration issue.

7.11.3 CI/CD Pipeline Security (GitHub Actions)

Status: ⚠ Weak - 16 GitHub Actions workflow files use overly broad `permissions: write-all` or implicit broad permissions; no pinned action SHA hashes.

`.github/workflows/` contains 11 active workflow files (plus `codeql-analysis.yml` and others). Several workflows use `write-all` permissions or do not explicitly restrict `permissions:` at the workflow level, granting write access to repository contents and packages by default. No workflow pins third-party actions to a commit SHA.

Security assessment

Overly broad workflow permissions mean a compromised or malicious GitHub Action in the dependency chain could push to the repository, publish packages, or access secrets. Pinning actions to commit SHA hashes and applying the principle of least-privilege permissions (`contents: read` as the default) are the two most impactful fixes here.

Relevant findings

- F-014 — Logging pipeline exposes raw log files; relevant to CI/CD audit trail integrity.
- F-018 — Error handler in production exposes internal paths that assist CI/CD environment enumeration.

7.11.4 Security Logging and Monitoring (Morgan + Winston)

Status: ⚠ Partial - Morgan combined-format logging and Winston application logging are enabled; but log files are exposed at an unauthenticated route, making the audit trail readable by any attacker.

`server.ts:338` mounts Morgan in combined format, logging all HTTP requests to stdout and to a file. `lib/logger.ts` configures Winston for application-level events. Both loggers are active by default.

Security assessment

The logging infrastructure works: requests and events are recorded. The critical gap is that the log files are readable without authentication at `GET /support/logs` (F-014 — Access logs exposed to unauthenticated users tamper potential). An attacker can read request logs to discover authenticated users, active sessions, and API call patterns - the audit trail becomes an intelligence source for the attacker rather than for the defender.

Relevant findings

- F-014 — Log files exposed at `/support/logs` without authentication; the audit trail is readable by unauthenticated attackers.
- F-018 — Error handler emits stack traces in production responses, enriching attacker intelligence.

7.11.5 Automated SCA scanning

Status: □ Adequate - GitHub CodeQL analysis workflow (`codeql-analysis.yml`) runs on push to master and on PRs; CodeQL covers JavaScript/TypeScript.

`.github/workflows/codeql-analysis.yml` runs GitHub's CodeQL static analysis on every push to `master` and on pull requests. The analysis covers JavaScript and TypeScript, which catches a subset of the injection and dangerous-eval findings. CodeQL's built-in queries detect `eval()` on user-controlled input and SQL concatenation patterns.

Security assessment

CodeQL provides a meaningful static analysis baseline. Its effectiveness is limited here because it is the only automated security gate and the findings it would surface (eval, SQL injection) are present in the current codebase - indicating either that the results are not reviewed or that the intentional vulnerability design suppresses alerts. The control is adequate as infrastructure; its operational value depends on a triage process that is outside the source tree.

Relevant findings

- F-014 — Log exposure is an operational finding not surfaced by static analysis.
- F-018 — Error-handler misconfiguration is a runtime finding; CodeQL would not catch it.

Additional cataloged controls without a dedicated subsection (no implementation prose and no linked findings): Automated dependency updates, Lockfile hygiene.

7.12 Real-time and Not Applicable Controls

`socket.io@3.1.x` is present in `package.json` and `lib/startup/registerWebsocketEvents.ts` registers WebSocket event handlers. No security finding was derived from the WebSocket or `Socket.IO` surface during this assessment.

Controls covered: 7.12.1 WebSocket Event Handler Security (`Socket.IO`)

- 7.12.1 WebSocket Event Handler Security (Socket.IO)

Implemented controls: `lib/startup/registerWebsocketEvents.ts` (`Socket.IO` event handler registration); `socket.io@3.1.x` (WebSocket server).

Assessment: The `Socket.IO` event handlers are present and functional. No injection, authentication bypass, or denial-of-service finding was identified on the WebSocket surface during this assessment. This section is marked Not Applicable - the WebSocket surface was reviewed and no controls are deficient.

7.12.1 WebSocket Event Handler Security (`Socket.IO`)

Status: - Not applicable - `Socket.IO` event handlers reviewed; no exploitable findings identified on this surface.

`lib/startup/registerWebsocketEvents.ts` registers `Socket.IO` event handlers for real-time features (order status updates, challenge notifications). The handlers receive client messages and broadcast server-side events. No user-controlled data is passed to dangerous sinks (eval, SQL, filesystem) via the WebSocket path.

Security assessment

The WebSocket surface was reviewed during reconnaissance and STRIDE analysis. The `Socket.IO` handlers do not introduce independent injection or privilege-escalation vectors beyond those already catalogued on the HTTP REST surface. No additional controls are required beyond the session/JWT controls described in §7.3.

Relevant findings

- No dedicated finding routed in this assessment.

7.13 Defense-in-Depth Summary

Verdict: □ Missing

Two individual controls provide genuine defence-in-depth value: the multi-stage distroless Docker image removes shell and package-manager access from the runtime container, and the CodeQL workflow catches a class of injection and eval patterns in CI. The RS256 algorithm choice for JWT signing is architecturally sound - the weakness is in the key management, not the algorithm. Morgan and Winston logging infrastructure is present and functional.

Restoring layered defence requires four structural repairs that each remove an independent exploitation path: (1) move all hardcoded secrets to runtime injection and rotate the RSA key pair, (2) replace the two raw `models.sequelize.query()` call sites with parameterized queries, (3) remove both `eval()` sinks and the `notevil` sandbox from the order processing path, and (4) remove every `bypassSecurityTrustHtml()` call and let Angular's default sanitizer run. Without these four, no browser-level or container-level hardening closes the Critical attack paths.

---

8. Threat Register

Findings are grouped by severity (Critical → High → Medium → Low); within a tier they are ordered by attack vektor (Repo-Read → Internet-Anon → Internet-User → Victim-Required). Each finding is a card with the same fixed fields, in order: **Severity · Component · Location → Issue → Root cause → Evidence → Fix → Classification** (with external CWE / OWASP links).

Risk Distribution: Critical: 9 · High: 16 · Medium: 6 · Low: 0 · **Total findings: 31**
STRIDE Coverage: Spoofing: 4 · Tampering: 12 · Repudiation: 1 · Information Disclosure: 8 · Denial of Service: 2 · Elevation of Privilege: 4

Findings index:

Critical (9)

F-002 · Hardcoded Cryptographic Key

Severity: Critical · **Component:** C-01 - Express REST API Backend · **Location:** `lib/insecurity.ts`:23

Issue: The RSA 2048-bit private key is embedded in `lib/insecurity.ts:23` and committed to a public GitHub repository. Any user can retrieve the key (git clone or GitHub web UI), call jwt.sign({id:1, email:'admin@juice-`sh.op`', isAdmin:true}, privateKey, {algorithm:'RS256'}) offline and produce a cryptographically valid admin JWT without any server interaction.

Root cause: Authentication can be circumvented or forged because credentials, signing keys, or password hashes are weak, missing, or exposed.

Evidence: ✓ verified - A signing key is embedded as a literal constant in source.

Fix: Move the cryptographic key out of source control into a managed secret store and rotate it → M-002 — Implement application-level CSP and remove bypassSecurityTrustHtml - Implement application-level CSP and remove bypassSecurityTrustHtml

Classification: Cryptographic Failures · CWE-321 · OWASP A02:2021 · walkthrough Walkthrough §3.2

F-001 · Improper Verification of Cryptographic Signature

Severity: Critical · **Component:** C-01 - Express REST API Backend · **Location:** `lib/insecurity.ts`:54

Issue: An attacker crafts a JWT with header {"alg":"none"} and empty signature. express-jwt 0.1.3 (routes via `lib/insecurity.ts:54`) accepts unsigned tokens because it does not enforce algorithm. Attacker sets isAdmin:true in the payload and gains full admin access. CVE-2015-9235.

Root cause: Authentication can be circumvented or forged because credentials, signing keys, or password hashes are weak, missing, or exposed.

Evidence: ✓ verified

```
// lib/insecurity.ts:54
  return str
}

export const isAuthorized = () => expressJwt(({ secret: publicKey }) as any)
export const denyAll = () => expressJwt({ secret: '' + Math.random() } as any)
export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h', algorithm:
        'RS256' })
export const verify = (token: string) => token ? (jws.verify as ((token: string, secret: string) =>
        boolean))(token, publicKey) : false
```

Fix: Pin the signature algorithm explicitly and reject `alg:none` and unknown algorithms → M-001 — Establish dependency vulnerability scanning and update SLA - Establish dependency vulnerability scanning and update SLA

Classification: Broken Authentication · CWE-347 · OWASP A07:2021 · walkthrough Walkthrough §3.1

F-003 · SQL Injection

Severity: Critical · **Component:** C-01 - Express REST API Backend · **Location:** `routes/login.ts`:34

Issue: `routes/login.ts:34` constructs an SQL query via template literal: SELECT * FROM Users WHERE email = '${ `req.body.email` }' AND password = '...' AND deletedAt IS NULL. An attacker submits email=admin@juice-`sh.op`'-- which comments out the password check, authenticating as admin without knowing the password. UNION-based injection can dump the entire Users table.

Root cause: User input flows into a server-side interpreter (SQL, NoSQL, XML, YAML, LDAP, OS shell) without parameterisation or schema validation.

Evidence: ✓ verified - SQL is assembled via string concatenation/interpolation of untrusted input.

```
// routes/login.ts:34

  return (req: Request, res: Response, next: NextFunction) => {
    verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
    models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '$
        {security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: tr
      .then((authenticatedUser) => { // vuln-code-snippet neutral-line loginAdminChallenge
        loginBenderChallenge loginJimChallenge
        const user = utils.queryResultToJson(authenticatedUser)
        if (user.data?.id && user.data.totpSecret !== '') {
```

Fix: Switch all SQL execution to parameterised queries or ORM-bound parameters → M-003 — Replace raw SQL with Sequelize parameterized query - Replace raw SQL with Sequelize parameterized query

Classification: Injection · CWE-89 · OWASP A03:2021 · walkthrough Walkthrough §3.3

F-004 · SQL Injection

Severity: Critical · **Component:** C-01 - Express REST API Backend · **Location:** `routes/search.ts`:23

Issue: `routes/search.ts:23` builds: SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' ...). An attacker submits q=')) UNION SELECT * FROM Users-- to enumerate all users. The same route at line 47 runs SELECT sql FROM sqlite_master on UNION success, exposing the full database schema.

Root cause: User input flows into a server-side interpreter (SQL, NoSQL, XML, YAML, LDAP, OS shell) without parameterisation or schema validation.

Evidence: ✓ verified - SQL is assembled via string concatenation/interpolation of untrusted input.

```
// routes/search.ts:23
  return (req: Request, res: Response, next: NextFunction) => {
    let criteria: any = req.query.q === 'undefined' ? '' : req.query.q ?? ''
    criteria = (criteria.length <= 200) ? criteria : criteria.substring(0, 200)
    models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE
        '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`) // vuln-code-snippet vuln-line unionSql
      .then(([products]: any) => {
        const dataString = JSON.stringify(products)
        if (challengeUtils.notSolved(challenges.unionSqlInjectionChallenge)) { // vuln-code-snippet hide-
        start
```

Fix: Switch all SQL execution to parameterised queries or ORM-bound parameters → M-004 — Use Sequelize Op.like with parameterized values for search - Use Sequelize Op.like with parameterized values for search

Classification: Injection · CWE-89 · OWASP A03:2021 · walkthrough Walkthrough §3.4

F-005 · Code Injection

Severity: Critical · **Component:** C-01 - Express REST API Backend · **Location:** `routes/b2bOrder.ts`:23

Issue: POST `/b2b/v2/orders` accepts orderLinesData (user-controlled string) and executes it via vm.runInContext('safeEval(orderLinesData)', sandbox). The notevil library has documented sandbox escape vulnerabilities. An attacker can execute arbitrary `Node.js` code by sending a crafted payload: {"orderLinesData":"require('child_process').execSync('id')"}.

Root cause: User-supplied data reaches a server-side code-execution sink (`eval`, sandbox primitives, deserialisation, prototype-pollution gadgets) and breaks out into arbitrary native execution.

Evidence: ✓ verified - User-supplied code is passed to a runtime evaluator without an allow-list of operations.

```
// routes/b2bOrder.ts:23
      try {
        const sandbox = { safeEval, orderLinesData }
        vm.createContext(sandbox)
        vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 })
        res.json({ cid: body.cid, orderNo: uniqueOrderNumber(), paymentDue: dateTwoWeeksFromNow() })
      } catch (err) {
        if (utils.getErrorMessage(err).match(/Script execution timed out.*/) != null) {
```

Fix: Replace runtime code generation (eval/Function/template render) with a data-only execution path → M-005 — Remove eval-based order processing; use structured input schema - Remove eval-based order processing; use structured input schema

Classification: Code Execution via Unsafe Deserialization or Eval · CWE-94 · OWASP A08:2021 · walkthrough Walkthrough §3.5

F-006 · Server-Side Template Injection

Severity: Critical · **Component:** C-01 - Express REST API Backend · **Location:** `routes/userProfile.ts`:62

Issue: Calls eval(code) where code is derived from the username field: const code = ... username ... When an authenticated user sets their username to a JavaScript expression (e.g. require('fs').readFileSync('`/etc/passwd`').`toString()`), the server evaluates it and the result becomes the displayed username. Full server-side JavaScript execution as the application process user.

Root cause: User-supplied data reaches a server-side code-execution sink (`eval`, sandbox primitives, deserialisation, prototype-pollution gadgets) and breaks out into arbitrary native execution.

Evidence: ✓ verified

```
// routes/userProfile.ts:62
        if (!code) {
          throw new Error('Username is null')
        }
        username = eval(code) // eslint-disable-line no-eval
      } catch (err) {
        username = '\\' + username
      }
```

Fix: Replace runtime code generation (eval/Function/template render) with a data-only execution path → M-006 — Remove eval() from username processing; use safe string rendering - Remove `eval()` from username processing; use safe string rendering

Classification: Code Execution via Unsafe Deserialization or Eval · CWE-95 · OWASP A08:2021 · walkthrough Walkthrough §3.6

F-027 · Password Hash with Insufficient Effort

Severity: Critical · **Component:** C-01 - Express REST API Backend · **Location:** `lib/insecurity.ts`:43

Issue: `models/user.ts:77` and `lib/insecurity.ts:43` hash passwords with a single-round MD5 with no salt: crypto.createHash('md5').update(data).digest('hex'). MD5 is a fast, cryptographically broken hash. The complete md5 rainbow table for common passwords fits in memory. An attacker who exfiltrates the Users table (via SQL injection, T-003 — SQL injection login authentication bypass or T-004 — SQL injection product search (UNION + schema exfil)) can reverse all passwords within hours using GPU-accelerated cracking or rainbow tables.

Root cause: Authentication can be circumvented or forged because credentials, signing keys, or password hashes are weak, missing, or exposed.

Evidence: ✓ verified - A non-iterating hash is used for password storage.

Fix: Replace the broken hash with a salted password-hashing function (bcrypt/Argon2id) → M-027 — Replace MD5 with bcrypt (cost factor 12+) for password hashing - Replace MD5 with bcrypt (cost factor 12+) for password hashing

Classification: Cryptographic Failures · CWE-916 · OWASP A02:2021 · walkthrough Walkthrough §3.9

F-019 · Cross-Site Scripting

Severity: Critical · **Component:** C-02 - Angular Single-Page Application · **Location:** `frontend/src/app/search-result/search-result.component.ts`:170

Issue: Calls this.sanitizer.bypassSecurityTrustHtml(queryParam) and binds the result to [innerHTML] in the template. An attacker crafts a URL with q=`<img src=x onerror=alert(document.cookie)>` and shares it. When the victim clicks the link, the Angular SPA renders the unsanitized parameter, executing the attacker's JavaScript in the victim's browser with full access to localStorage (JWT token).

Root cause: Attacker-controlled content is rendered in the victim's browser without sanitisation; combined with session tokens held in JavaScript-readable storage, any payload yields immediate account takeover.

Evidence: ✓ verified - User input is rendered as HTML without contextual output encoding.

```
// frontend/src/app/search-result/search-result.component.ts:170
        this.io.socket().emit('verifyLocalXssChallenge', queryParam)
      }) // vuln-code-snippet hide-end
      this.dataSource.filter = queryParam.toLowerCase()
      this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam) // vuln-code-snippet vuln-
        line localXssChallenge xssBonusChallenge
      this.gridDataSource.subscribe((result: any) => {
        if (result.length === 0) {
          this.emptyState = true
```

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → M-019 — Remove bypassSecurityTrustHtml; use Angular's default sanitization - Remove bypassSecurityTrustHtml; use Angular's default sanitization

Classification: Cross-Site Scripting (XSS) · CWE-79 · OWASP A03:2021 · walkthrough Walkthrough §3.7

F-020 · Cross-Site Scripting

Severity: Critical · **Component:** C-02 - Angular Single-Page Application · **Location:** `frontend/src/app/administration/administration.component.ts`:78

Issue: `frontend/src/app/administration/administration.component.ts:78` marks `feedback.comment` as trusted HTML via bypassSecurityTrustHtml(`feedback.comment`). The admin panel at `/administration` renders these comments via [innerHTML] binding. Any user

who submits feedback with XSS payload stores it in the database; when admin views the panel, the payload executes in the admin's browser, potentially stealing the admin JWT and enabling privilege escalation.

Root cause: Attacker-controlled content is rendered in the victim's browser without sanitisation; combined with session tokens held in JavaScript-readable storage, any payload yields immediate account takeover.

Evidence: ✓ verified - User input is rendered as HTML without contextual output encoding.

```
// frontend/src/app/administration/administration.component.ts:78
      next: (feedbacks) => {
        this.feedbackDataSource = feedbacks
        for (const feedback of this.feedbackDataSource) {
          feedback.comment = this.sanitizer.bypassSecurityTrustHtml(feedback.comment)
        }
        this.feedbackDataSource = new MatTableDataSource(this.feedbackDataSource)
        this.feedbackDataSource.paginator = this.paginatorFeedb
```

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → M-020 — Sanitize feedback content server-side before storage; remove bypassSecurityTrustHtml in admin panel - Sanitize feedback content server-side before storage; remove bypassSecurityTrustHtml in admin panel

Classification: Cross-Site Scripting (XSS) · CWE-79 · OWASP A03:2021 · walkthrough Walkthrough §3.8

□ High (16)

F-007 · XML External Entity (XXE)

Severity: □ High · **Component:** C-01 - Express REST API Backend · **Location:** `routes/fileUpload.ts`:83

Issue: Parses uploaded XML with libxmljs2 using { noent: true } which enables external entity resolution. An attacker uploads an XML file containing . Same mechanism can trigger SSRF to internal services.

Root cause: User input flows into a server-side interpreter (SQL, NoSQL, XML, YAML, LDAP, OS shell) without parameterisation or schema validation.

Evidence: ✓ verified - An XML parser is configured to expand external entities while processing untrusted input.

```
// routes/fileUpload.ts:83
        const sandbox = { libxml, data }
        vm.createContext(sandbox)
        const xmlDoc = vm.runInContext('libxml.parseXml(data, { noblanks: true, noent: true, nocdata:
        true })', sandbox, { timeout: 2000 })
        const xmlString = xmlDoc.toString(false)
        challengeUtils.solveIf(challenges.xxeFileDisclosureChallenge, () => { return
        (utils.matchesEtcPasswdFile(xmlString) || utils.matchesSystemIniFile(xmlString)) })
```

Fix: Disable external entity resolution on every XML parser and reject DOCTYPE declarations → M-007 — Set noent:false in libxmljs2 parsing to disable external entity resolution - Set `noent:false` in libxmljs2 parsing to disable external entity resolution

Classification: Injection · CWE-611 · OWASP A03:2021

F-008 · Path Traversal

Severity: □ High · **Component:** C-01 - Express REST API Backend · **Location:** `routes/fileUpload.ts`:42

Issue: `routes/fileUpload.ts:42-45` extracts ZIP files without path sanitization: const absolutePath = path.resolve('uploads/complaints/' + fileName). An attacker crafts a ZIP containing an entry with path ../../`routes/login.js` and uploads it. The server extracts to an arbitrary location outside the intended directory, potentially overwriting application files.

Root cause: Confidential files, credentials, and management-plane endpoints are reachable on unauthenticated routes; SSRF lets the server fetch internal resources on the attacker's behalf; unsafe path-handling primitives leak server content.

Evidence: ✓ verified - User-supplied path components are joined without traversal-safe canonicalisation.

```
// routes/fileUpload.ts:42
              .on('entry', function (entry: any) {
                const fileName = entry.path
                const absolutePath = path.resolve('uploads/complaints/' + fileName)
                challengeUtils.solveIf(challenges.fileWriteChallenge, () => { return absolutePath ===
        path.resolve('ftp/legal.md') })
                if (absolutePath.includes(path.resolve('.'))) {
```

Fix: Resolve and normalise every constructed path and reject anything that escapes the intended base directory → M-008 — Validate extracted paths against target directory before writing - Validate extracted paths against target directory before writing

Classification: Insecure File Handling · CWE-22 · OWASP A04:2021

F-010 · Information Disclosure

Severity: □ High · **Component:** C-01 - Express REST API Backend · **Location:** `server.ts`:281

Issue: Exposes three unauthenticated file-serving routes: /ftp/ (directory listing of FTP files), /encryptionkeys/ (RSA public key + others), /support/logs/ (raw application access logs). Log files contain IP addresses, user actions, and authentication tokens in the morgan combined format. Any internet user can read application logs at `/support/logs/access.log`.

Root cause: Confidential files, credentials, and management-plane endpoints are reachable on unauthenticated routes; SSRF lets the server fetch internal resources on the attacker's behalf; unsafe path-handling primitives leak server content.

Evidence: ✓ verified - Sensitive runtime values are served unauthenticated to any caller.

Fix: Restrict the response to the minimum fields needed and never echo secrets → M-010 — Add authentication middleware to sensitive file serving routes - Add authentication middleware to sensitive file serving routes

Classification: Error Information Disclosure · CWE-200 · OWASP A05:2021

F-016 · Missing Rate Limiting (Brute-Force)

Severity: □ High · **Component:** C-01 - Express REST API Backend · **Location:** `server.ts`:594

Issue: POST `/rest/user/login` has no rate limiting. An attacker can attempt credential stuffing or brute-force attacks at unlimited speed. A 1 million-entry credential list can be tested in minutes from a single IP.

Evidence: ✓ verified

```
// server.ts:594

  /* Custom Restful API */
  app.post('/rest/user/login', login())
  app.get('/rest/user/change-password', changePassword())
  app.post('/rest/user/reset-password', resetPassword())
```

Fix: Apply rate limiting and lock-out thresholds on authentication endpoints → M-016 — Add express-rate-limit to login endpoint (already in dependencies) - Add express-rate-limit to login endpoint (already in dependencies)

Classification: Denial of Service · CWE-307 · OWASP A04:2021

F-022 · ~~Insecure Storage of Sensitive Information~~ ⚠

Severity: □ High · **Component:** C-02 - Angular Single-Page Application · **Location:** `frontend/src/app/Services/authentication.service.ts`:1

Issue: The Angular SPA stores JWT tokens in localStorage (accessible via JavaScript). Any successful XSS payload (T-019 — DOM XSS search results rendered via trust HTML bypass, T-020 — Stored XSS feedback comments rendered unsanitized in admin panel, T-021 — XSS

last login ip rendered as trusted HTML) can read document.localStorage.getItem('token') and exfiltrate the JWT to an attacker-controlled server. Since JWTs last 6 hours and there is no revocation, the attacker has a 6-hour session window.

Root cause: Attacker-controlled content is rendered in the victim's browser without sanitisation; combined with session tokens held in JavaScript-readable storage, any payload yields immediate account takeover.

Evidence: ⚠ refuted

Fix: M-022 — Store JWT in HttpOnly cookie to prevent XSS-based theft - Store JWT in HttpOnly cookie to prevent XSS-based theft

Classification: Insecure Client-Side Storage · CWE-922 · OWASP A02:2021

F-023 · Angular route guards bypassable client

Severity: □ High · **Component:** C-02 - Angular Single-Page Application · **Location:** `frontend/src/app/app.guard.ts`:12

Issue: `frontend/src/app/app.guard.ts` provides LoginGuard and AdminGuard for client-side routing. Because these guards only check localStorage for a token, an attacker who forges a valid JWT (T-001 — JWT algorithm confusion algorithm none bypass, T-002 — RSA private key hardcoded offline JWT forgery) or manually sets a crafted token in localStorage can bypass the client-side guard. Even without token forgery, route guards are client-side only - direct API calls bypass them entirely.

Evidence: ✓ verified

Fix: M-023 — Enforce authorization server-side; client guards are UI-only - Enforce authorization server-side; client guards are UI-only

Classification: Broken Access Control · CWE-602 · OWASP A01:2021

F-028 · Information Disclosure

Severity: □ High · **Component:** C-03 - Data Layer (SQLite + MarsDB) · **Location:** `models/user.ts`:28

Issue: The User Sequelize model has no field exclusion by default. REST endpoints backed by Sequelize (via finale-rest) may return all model attributes including password (MD5 hash) and totpSecret in API responses. An attacker who can trigger a user data endpoint (e.g. GET `/api/Users/:id`) receives the password hash which can be cracked offline.

Root cause: Confidential files, credentials, and management-plane endpoints are reachable on unauthenticated routes; SSRF lets the server fetch internal resources on the attacker's behalf; unsafe path-handling primitives leak server content.

Evidence: ✓ verified - Sensitive runtime values are served unauthenticated to any caller.

Fix: Restrict the response to the minimum fields needed and never echo secrets → M-028 — Exclude password and totpSecret from all API serialization - Exclude password and totpSecret from all API serialization

Classification: Error Information Disclosure · CWE-200 · OWASP A05:2021

F-030 · Information Disclosure

Severity: □ High · **Component:** C-03 - Data Layer (SQLite + MarsDB) · **Location:** `routes/search.ts`:47

Issue: `routes/search.ts:47` executes SELECT sql FROM sqlite_master when the UNION injection challenge is solved. This reveals the complete SQLite schema including all table names, column names, and indexes. Even if the direct injection is mitigated, the pattern of exposing schema metadata is an information disclosure that aids further attacks.

Root cause: Confidential files, credentials, and management-plane endpoints are reachable on unauthenticated routes; SSRF lets the server fetch internal resources on the attacker's behalf; unsafe path-handling primitives leak server content.

Evidence: ✓ verified - Sensitive runtime values are served unauthenticated to any caller.

Fix: Restrict the response to the minimum fields needed and never echo secrets → M-030 — Mitigate SQL injection (T-004) eliminates schema exposure - Mitigate SQL injection (T-004 — SQL injection product search (UNION + schema exfil)) eliminates schema exposure

Classification: Error Information Disclosure · CWE-200 · OWASP A05:2021

F-009 · Server-Side Request Forgery (SSRF)

Severity: □ High · **Component:** C-01 - Express REST API Backend · **Location:** `routes/profileImageUrlUpload.ts`:24

Issue: `routes/profileImageUrlUpload.ts:24` fetches a user-controlled URL via fetch(url). An authenticated attacker provides url=http://169.254.169.254/latest/meta-data/ (AWS metadata service) or url=http://internal-service:8080/admin. The server makes the request and may expose the response or trigger actions on internal services.

Root cause: Confidential files, credentials, and management-plane endpoints are reachable on unauthenticated routes; SSRF lets the server fetch internal resources on the attacker's behalf; unsafe path-handling primitives leak server content.

Evidence: ✓ verified - An outbound request is issued to a URL derived from untrusted input without an allow-list.

```
// routes/profileImageUrlUpload.ts:24
      if (loggedInUser) {
        try {
          const response = await fetch(url)
          if (!response.ok || !response.body) {
            throw new Error('url returned a non-OK status code or an empty body')
```

Fix: Validate the URL scheme + host against an explicit allow-list before issuing outbound requests → M-009 — Implement URL allowlist or SSRF protection library for outbound fetch - Implement URL allowlist or SSRF protection library for outbound fetch

Classification: Server-Side Request Forgery · CWE-918 · OWASP A10:2021

F-011 · Insecure Direct Object Reference (IDOR)

Severity: □ High · **Component:** C-01 - Express REST API Backend · **Location:** `routes/basket.ts`:19

Issue: GET `/rest/basket/:id` retrieves a basket by numeric ID. The route applies `security.isAuthorized()` but does NOT check that the basket belongs to the requesting user. An attacker authenticated as any user can enumerate basket IDs (e.g., GET `/rest/basket/1` through `/rest/basket/100`) and view other users' baskets, including saved payment methods.

Root cause: Authorisation checks are absent or bypassable, allowing horizontal and vertical privilege jumps from a self-registered or low-rights account. Includes mass-assignment of privileged attributes.

Evidence: ✓ verified - An object-identity parameter is trusted from the request without server-side ownership check.

```
// routes/basket.ts:19
    try {
      const id = req.params.id
      const basket = await BasketModel.findOne({ where: { id }, include: [{ model: ProductModel,
        paranoid: false, as: 'Products' }] })
      /* jshint eqeqeq:false */
      challengeUtils.solveIf(challenges.basketAccessChallenge, () => {
```

Fix: Tie every object lookup to the requesting user's identity and reject cross-tenant references → M-011 — Validate basket ownership against authenticated user's ID from JWT - Validate basket ownership against authenticated user's ID from JWT

Classification: Broken Access Control · CWE-639 · OWASP A01:2021

F-015 · Path Traversal

Severity: □ High · **Component:** C-01 - Express REST API Backend · **Location:** `routes/dataErasure.ts`:69

Issue: Uses path.resolve(`req.body.layout`).`toLowerCase()` with the result being used to read a file. While `path.resolve()` normalizes the path, it resolves relative to the process working directory. A crafted layout value (e.g. ../..`/server`.ts) can read arbitrary files from the server's filesystem.

Root cause: Confidential files, credentials, and management-plane endpoints are reachable on unauthenticated routes; SSRF lets the server fetch internal resources on the attacker's behalf; unsafe path-handling primitives leak server content.

Evidence: ✓ verified - User-supplied path components are joined without traversal-safe canonicalisation.

```
// routes/dataErasure.ts:69
    res.clearCookie('token')
    if (req.body.layout) {
      const filePath: string = path.resolve(req.body.layout).toLowerCase()
      const isForbiddenFile: boolean = (filePath.includes('ftp') || filePath.includes('ctf.key') ||
        filePath.includes('encryptionkeys'))
      if (!isForbiddenFile) {
```

Fix: Resolve and normalise every constructed path and reject anything that escapes the intended base directory → M-015 — Whitelist permitted layout file names; use path.basename() to strip directory traversal - Whitelist permitted layout file names; use `path.basename()` to strip directory traversal

Classification: Insecure File Handling · CWE-22 · OWASP A04:2021

F-017 · Mass assignment wallet balance manipulable

Severity: □ High · **Component:** C-01 - Express REST API Backend · **Location:** `routes/wallet.ts`:12

Issue: `routes/wallet.ts:12` fetches wallet using req.body.UserId: WalletModel.findOne({ where: { UserId: req.body.UserId } }). An authenticated attacker can set req.body.UserId to any other user's ID to view or increment their wallet balance. `routes/address.ts:11` has the same pattern for addresses.

Root cause: Authorisation checks are absent or bypassable, allowing horizontal and vertical privilege jumps from a self-registered or low-rights account. Includes mass-assignment of privileged attributes.

Evidence: ✓ verified - Mass assignment is enabled because the model accepts request fields wholesale.

```
// routes/wallet.ts:12
export function getWalletBalance () {
  return async (req: Request, res: Response, next: NextFunction) => {
    const wallet = await WalletModel.findOne({ where: { UserId: req.body.UserId } })
    if (wallet != null) {
      res.status(200).json({ status: 'success', data: wallet.balance })
```

Fix: M-017 — Always derive UserId from verified JWT payload, never from request body - Always derive UserId from verified JWT payload, never from request body

Classification: Broken Access Control · CWE-915 · OWASP A01:2021

F-026 · NoSQL Injection

Severity: □ High · **Component:** C-03 - Data Layer (SQLite + MarsDB) · **Location:** `routes/showProductReviews.ts`:36

Issue: `routes/showProductReviews.ts:36` queries db.reviewsCollection.find({ $where: '`this.product` == ' + id }). An attacker supplies id=1; return true - the $where JavaScript expression evaluates to true for all documents, returning all reviews regardless of product. SSJI (Server-Side JavaScript Injection) in MarsDB.

Root cause: User input flows into a server-side interpreter (SQL, NoSQL, XML, YAML, LDAP, OS shell) without parameterisation or schema validation.

Evidence: ✓ verified

```
// routes/showProductReviews.ts:36
    const t0 = new Date().getTime()

    db.reviewsCollection.find({ $where: 'this.product == ' + id }).then((reviews: Review[]) => {
      const t1 = new Date().getTime()
      challengeUtils.solveIf(challenges.noSqlCommandChallenge, () => { return (t1 - t0) > 2000 })
```

Fix: Replace string concatenation in query operators with parameter binding → M-026 — Replace $where JavaScript expression with standard MarsDB equality query - Replace $where JavaScript expression with standard MarsDB equality query

Classification: Injection · CWE-943 · OWASP A03:2021

F-031 · Uncontrolled Resource Consumption

Severity: □ High · **Component:** C-03 - Data Layer (SQLite + MarsDB) · **Location:** `routes/showProductReviews.ts`:36

Issue: The $where JavaScript expression executed by MarsDB (T-026 — NoSQL injection via $where operator in MarsDB reviews vector) can include `process.exit()` or throw new `Error()` calls that crash or destabilize the `Node.js` process. Since MarsDB is an in-memory store and the application runs as a single process, a DoS via crafted review query brings down the entire application.

Evidence: ✓ verified

Fix: Bound the request rate and the per-request resource budget on this endpoint → M-031 — Fix NoSQL injection T-026 to eliminate this DoS vector - Fix NoSQL injection T-026 — NoSQL injection via $where operator in MarsDB reviews to eliminate this DoS vector

Classification: Denial of Service · CWE-400 · OWASP A04:2021

F-021 · Cross-Site Scripting

Severity: □ High · **Component:** C-02 - Angular Single-Page Application · **Location:** `frontend/src/app/last-login-ip/last-login-ip.component.ts`:39

Issue: `frontend/src/app/last-login-ip/last-login-ip.component.ts:39` wraps the server-provided lastLoginIp in bypassSecurityTrustHtml. If the last-login IP is set to attacker-controlled value (e.g. through an injection point), it renders as HTML in the user's own profile page. Server-side request with X-Forwarded-For: `<script>`alert(1)`</script>` could inject into this field.

Root cause: Attacker-controlled content is rendered in the victim's browser without sanitisation; combined with session tokens held in JavaScript-readable storage, any payload yields immediate account takeover.

Evidence: ✓ verified - User input is rendered as HTML without contextual output encoding.

```
// frontend/src/app/last-login-ip/last-login-ip.component.ts:39
      if (payload.data.lastLoginIp) {

        this.lastLoginIp = this.sanitizer.bypassSecurityTrustHtml(`<small>${payload.data.lastLoginIp}</
        small>`)
      }
    }
```

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → M-021 — Remove bypassSecurityTrustHtml; render IP as plain text - Remove bypassSecurityTrustHtml; render IP as plain text

Classification: Cross-Site Scripting (XSS) · CWE-79 · OWASP A03:2021

F-025 · Cross-Site Scripting

Severity: □ High · **Component:** C-02 - Angular Single-Page Application · **Location:** `frontend/src/app/search-result/search-result.component.ts`:132

Issue: `frontend/src/app/search-result/search-result.component.ts:132` and `product-details/product-details.component.html:16` use [innerHTML] for product descriptions. If an admin or data seeding creates a product with XSS in the description (or reviews), it executes for every user viewing that product. The application seeds products from data/static files, and any tampered seed data propagates to all users.

Root cause: Attacker-controlled content is rendered in the victim's browser without sanitisation; combined with session tokens held in JavaScript-readable storage, any payload yields immediate account takeover.

Evidence: ✓ verified - User input is rendered as HTML without contextual output encoding.

```
// frontend/src/app/search-result/search-result.component.ts:132
  trustProductDescription (tableData: any[]) { // vuln-code-snippet neutral-line restfulXssChallenge
    for (let i = 0; i < tableData.length; i++) { // vuln-code-snippet neutral-line restfulXssChallenge
      tableData[i].description = this.sanitizer.bypassSecurityTrustHtml(tableData[i].description) //
        vuln-code-snippet vuln-line restfulXssChallenge
    } // vuln-code-snippet neutral-line restfulXssChallenge
  } // vuln-code-snippet neutral-line restfulXssChallenge
```

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → M-025 — Sanitize product descriptions server-side; use textContent for display - Sanitize product descriptions server-side; use textContent for display

Classification: Cross-Site Scripting (XSS) · CWE-79 · OWASP A03:2021

Medium (6)

F-013 · Information Disclosure

Severity: Medium · **Component:** C-01 - Express REST API Backend · **Location:** `server.ts`: 718

Issue: GET `/metrics` serves prom-client metrics without any authentication middleware. These metrics expose request counts per route, response time histograms, `Node.js` heap size, event loop lag, active handles, and file descriptor counts. An attacker can fingerprint the server version, infer traffic patterns, and identify low-traffic admin routes.

Root cause: Confidential files, credentials, and management-plane endpoints are reachable on unauthenticated routes; SSRF lets the server fetch internal resources on the attacker's behalf; unsafe path-handling primitives leak server content.

Evidence: ✓ verified

Fix: Restrict the response to the minimum fields needed and never echo secrets → M-013 — Add authentication or restrict /metrics to internal network - Add authentication or restrict `/metrics` to internal network

Classification: Error Information Disclosure · CWE-200 · OWASP A05:2021

F-014 · Insufficient Logging

Severity: Medium · **Component:** C-01 - Express REST API Backend · **Location:** `server.ts`: 282

Issue: Application logs are served at /support/logs/ without authentication. Beyond disclosure (T-010 — Sensitive files and logs exposed without authentication), an attacker can observe log file names, access patterns, and timestamps before performing attacks, then confirm whether their actions appeared in logs. This makes it trivial to verify whether a specific attack was logged.

Evidence: ✓ verified

Fix: M-014 — Restrict log access; send logs to separate log aggregation system - Restrict log access; send logs to separate log aggregation system

Classification: Missing Audit Logging & Accountability · CWE-778 · OWASP A09:2021

F-018 · Error Message Disclosure

Severity: □ Medium · **Component:** C-01 - Express REST API Backend · **Location:** `server.ts`:1

Issue: The errorhandler npm package is imported (`package.json`) and used in development mode. Express next(err) calls propagate to this handler, which returns full stack traces, file paths, and line numbers in HTTP responses. An attacker triggering any route error (e.g. malformed JSON, SQL errors) receives detailed implementation information.

Root cause: Confidential files, credentials, and management-plane endpoints are reachable on unauthenticated routes; SSRF lets the server fetch internal resources on the attacker's behalf; unsafe path-handling primitives leak server content.

Evidence: ◌ ambiguous

Fix: Replace developer error pages with a generic message in production responses → M-018 — Remove errorhandler in production; use generic error responses - Remove errorhandler in production; use generic error responses

Classification: Error Information Disclosure · CWE-209 · OWASP A05:2021

F-029 · Insecure Direct Object Reference (IDOR)

Severity: □ Medium · **Component:** C-03 - Data Layer (SQLite + MarsDB) · **Location:** `routes/updateProductReviews.ts`:1

Issue: PATCH `/rest/products/reviews` at `server.ts:634` applies `security.isAuthorized()` but `routes/updateProductReviews.ts` does not verify that the authenticated user authored the review. Any authenticated user can update any review by knowing the review's _id, effectively defacing product reviews under other users' names.

Root cause: Authorisation checks are absent or bypassable, allowing horizontal and vertical privilege jumps from a self-registered or low-rights account. Includes mass-assignment of privileged attributes.

Evidence: ◌ ambiguous

Fix: Tie every object lookup to the requesting user's identity and reject cross-tenant references → M-029 — Verify review author matches authenticated user before allowing update - Verify review author matches authenticated user before allowing update

Classification: Broken Access Control · CWE-639 · OWASP A01:2021

F-012 · Open Redirect

Severity: □ Medium · **Component:** C-01 - Express REST API Backend · **Location:** `routes/redirect.ts`:19

Issue: GET `/redirect`?to= calls security.isRedirectAllowed(toUrl) which checks if the URL starts with a known allowlist entry (e.g. https://www.`paypal.com`). An attacker can bypass this with https://www.`paypal.com.attacker.com` or by appending query params. Users are redirected to attacker-controlled sites (phishing), and the redirect appears to originate from the trusted juice-shop domain.

Root cause: Confidential files, credentials, and management-plane endpoints are reachable on unauthenticated routes; SSRF lets the server fetch internal resources on the attacker's behalf; unsafe path-handling primitives leak server content.

Evidence: ✓ verified

Fix: M-012 — Use exact hostname matching instead of prefix matching for redirect allowlist - Use exact hostname matching instead of prefix matching for redirect allowlist

Classification: Open Redirect · CWE-601 · OWASP A01:2021

F-024 · Cross-Site Scripting

Severity: □ Medium · **Component:** C-02 - Angular Single-Page Application · **Location:** `frontend/src/hacking-instructor/challenges/reflectedXss.ts`:77

Issue: Multiple files in frontend/src/hacking-instructor/challenges/ use postMessage/iframe. `frontend/src/hacking-instructor/challenges/domXss.ts:93`,99 and `reflectedXss.ts:77` demonstrate postMessage payloads. A malicious parent page can post crafted messages to iframes embedding the application, potentially triggering DOM XSS if message origin is not validated.

Root cause: Attacker-controlled content is rendered in the victim's browser without sanitisation; combined with session tokens held in JavaScript-readable storage, any payload yields immediate account takeover.

Evidence: ✓ verified

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → M-024 — Validate postMessage origin before processing; add frame-ancestors CSP directive - Validate postMessage origin before processing; add frame-ancestors CSP directive

Classification: Cross-Site Scripting (XSS) · CWE-79 · OWASP A03:2021

Evidence verification: rows tagged `⚠ (evidence refuted)` were re-checked by the Phase 10a evidence-verifier (see `.evidence-verification.json`) and the cited `file:line` did **not** show the claimed weakness. Their raw severity is preserved, but chain-elevation has been

suppressed by the triage stage. Rows tagged `◌ (evidence ambiguous)` could not be confirmed or refuted from the cited snippet alone - a human reviewer should decide whether to keep, downgrade, or remove these findings.

---

9. Mitigation Register

Each mitigation block lists the findings it **Addresses**, the CWEs it **Prevents**, and the **Priority** (P1 = before deployment, P2 = current sprint, P3 = next quarter, P4 = backlog). The **Why** / **How** / **Verification** fields are populated only when authored; if a field is omitted, refer to the linked finding's Evidence line for file:line context and to the threat-category description in §8 Threat Register for the underlying weakness.

Mitigations index:
M-001 — Establish dependency vulnerability scanning and update SLA
M-002 — Implement application-level CSP and remove bypassSecurityTrustHtml
M-003 — Replace raw SQL with Sequelize parameterized query
M-004 — Use Sequelize Op.like with parameterized values for search
M-005 — Remove eval-based order processing; use structured input schema
M-006 — Remove eval() from username processing; use safe string rendering
M-007 — Set noent:false in libxmljs2 parsing to disable external entity resolution
M-008 — Validate extracted paths against target directory before writing
M-009 — Implement URL allowlist or SSRF protection library for outbound fetch
M-010 — Add authentication middleware to sensitive file serving routes
M-011 — Validate basket ownership against authenticated user's ID from JWT
M-012 — Use exact hostname matching instead of prefix matching for redirect allowlist
M-013 — Add authentication or restrict /metrics to internal network
M-014 — Restrict log access; send logs to separate log aggregation system
M-015 — Whitelist permitted layout file names; use path.basename() to strip directory traversal
M-016 — Add express-rate-limit to login endpoint (already in dependencies)
M-017 — Always derive UserId from verified JWT payload, never from request body
M-018 — Remove errorhandler in production; use generic error responses
M-019 — Remove bypassSecurityTrustHtml; use Angular's default sanitization
M-020 — Sanitize feedback content server-side before storage; remove bypassSecurityTrustHtml in admin panel
M-021 — Remove bypassSecurityTrustHtml; render IP as plain text
M-022 — Store JWT in HttpOnly cookie to prevent XSS-based theft
M-023 — Enforce authorization server-side; client guards are UI-only
M-024 — Validate postMessage origin before processing; add frame-ancestors CSP directive
M-025 — Sanitize product descriptions server-side; use textContent for display
M-026 — Replace $where JavaScript expression with standard MarsDB equality query
M-027 — Replace MD5 with bcrypt (cost factor 12+) for password hashing
M-028 — Exclude password and totpSecret from all API serialization
M-029 — Verify review author matches authenticated user before allowing update
M-030 — Mitigate SQL injection (T-004) eliminates schema exposure
M-031 — Fix NoSQL injection T-026 to eliminate this DoS vector

P1 — Immediate

M-002 — Implement application-level CSP and remove bypassSecurityTrustHtml

Addresses:

- F-002 — RSA private key hardcoded offline JWT forgery - RSA private key hardcoded offline JWT forgery

Priority: P1 - Immediate · **Effort:** Medium · **File:** `lib/insecurity.ts:23`

How:

1. Remove the hardcoded private key from `lib/insecurity.ts`
2. Generate a new RSA key pair: `openssl genrsa -out jwt_private.pem` 2048
3. Store in environment variable: `process.env.JWT_PRIVATE_KEY`
4. Load at runtime: const `privateKey` = `process.env.JWT_PRIVATE_KEY`

```
const privateKey = process.env.JWT_PRIVATE_KEY;
if (!privateKey) throw new Error('JWT_PRIVATE_KEY not set');
```

Reference: https:// `cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html`

M-003 — Replace raw SQL with Sequelize parameterized query

Addresses:

- F-003 — SQL injection login authentication bypass - SQL injection login authentication bypass

Priority: P1 - Immediate · **Effort:** Low · **File:** `routes/login.ts:34`

How:

1. Replace `models.sequelize.query(template literal)` with `UserModel.findOne({ where: { email: req.body.email, password: hash, deletedAt: null } })`
2. Let Sequelize handle parameterization automatically

```
const user = await UserModel.findOne({
  where: { email: req.body.email, password: security.hash(req.body.password), deletedAt: null }
});
```

Reference: https://sequelize.org/docs/v6/core-concepts/model-querying-basics/

M-004 — Use Sequelize Op.like with parameterized values for search

Addresses:

- F-004 — SQL injection product search (UNION + schema exfil) - SQL injection product search (UNION + schema exfil)

Priority: P1 - Immediate · **Effort:** Low · **File:** `routes/search.ts:23`

How:

1. Replace raw query with Sequelize where clause using Op.like
2. Use replacements parameter if raw query is needed

```
const products = await ProductModel.findAll({
  where: { [Op.or]: [{ name: { [Op.like]: `%${criteria}%` } }, { description: { [Op.like]: `%${criteria}
        %` } }], deletedAt: null }
});
```

Reference: https://sequelize.org/docs/v6/other-topics/querying/#operators

---

M-005 — Remove eval-based order processing; use structured input schema

Addresses:

- F-005 — Remote code execution notevil sandbox bypass in B2B API - Remote code execution notevil sandbox bypass in B2B API

Priority: P1 - Immediate · **Effort:** Medium · **File:** `routes/b2bOrder.ts:23`

How:

1. Replace `orderLinesData` eval with a structured JSON schema (e.g. array of {sku, qty, price})
2. Validate with express-validator or joi
3. Remove notevil dependency

```
const orderSchema = Joi.array().items(Joi.object({ sku: Joi.string().required(), qty:
        Joi.number().integer().min(1).required() }));
const { error, value } = orderSchema.validate(req.body.orderLines);
```

Reference: https:// `cwe.mitre.org/data/definitions/94.html`

M-006 — Remove eval() from username processing; use safe string rendering

Addresses:

- F-006 — Server side code injection via eval on username field - Server side code injection via eval on username field

Priority: P1 - Immediate · **Effort:** Low · **File:** `routes/userProfile.ts:62`

How:

1. Remove the `eval(code)` call entirely
2. Use the username string directly without evaluation
3. Apply HTML entity encoding before rendering

```
// Before: username = eval(code)
// After: username = sanitizeHtml(user.username, { allowedTags: [] })
```

Reference: https://`cwe.mitre.org/data/definitions/95.html`

M-019 — Remove bypassSecurityTrustHtml; use Angular's default sanitization

Addresses:

- F-019 — DOM XSS search results rendered via trust HTML bypass - DOM XSS search results rendered via trust HTML bypass

Priority: P1 - Immediate · **Effort:** Low · **File:** `frontend/src/app/search-result/search-result.component.ts:170`

How:

1. Remove all calls to `bypassSecurityTrustHtml()` in `search-result.component.ts`
2. Display the search query as plain text using {{ `searchValue` }} instead of [`innerHTML`]
3. Audit all 14 `bypassSecurityTrustHtml` callsites

```
// Before: this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam)
// After: this.searchValue = queryParam  // rendered via {{ searchValue }} in template
```

Reference: https://angular.io/guide/security#xss

M-020 — Sanitize feedback content server-side before storage; remove bypassSecurityTrustHtml in admin panel

Addresses:

- F-020 — Stored XSS feedback comments rendered unsanitized in admin panel - Stored XSS feedback comments rendered unsanitized in admin panel

Priority: P1 - Immediate · **Effort:** Medium · **File:** `frontend/src/app/administration/administration.component.ts:78`

How:

1. Sanitize feedback on submission with DOMPurify or sanitize-html (properly configured)
2. Remove `bypassSecurityTrustHtml` in `administration.component.ts:60`,78
3. Use [`textContent`] instead of [`innerHTML`] for user-submitted content

```
// Never call bypassSecurityTrust*; let Angular sanitize.
// template:  <div [innerHTML]="product.description"></div>
// component: no DomSanitizer.bypassSecurityTrustHtml(...)
this.product.description = raw  // bound directly; Angular escapes
```

Verification: Insert `<img src=x onerror=alert(1)>` via the affected field and confirm the browser renders the escaped text, not an alert dialog.

Reference: https:// `cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html`

M-027 — Replace MD5 with bcrypt (cost factor 12+) for password hashing

Addresses:

- F-027 — MD5 password hashing passwords trivially reversible - MD5 password hashing passwords trivially reversible

Priority: P1 - Immediate · **Effort:** Medium · **File:** `lib/insecurity.ts:43`

How:

1. Install bcrypt: `npm install bcrypt @types/bcrypt`
2. Replace `security.hash()` with `bcrypt.hash(password, 12)` for new passwords
3. Update login to use `bcrypt.compare(plaintext, hash)`
4. Migrate existing passwords: on next login, verify MD5, then re-hash with bcrypt

```
import bcrypt from 'bcrypt';
const SALT_ROUNDS = 12;
export const hashPassword = (p: string) => bcrypt.hash(p, SALT_ROUNDS);
export const verifyPassword = (p: string, h: string) => bcrypt.compare(p, h);
```

Reference: https:// `cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`

P2 — This Sprint

M-001 — Establish dependency vulnerability scanning and update SLA

Addresses:

- F-001 — JWT algorithm confusion algorithm none bypass - JWT algorithm confusion algorithm none bypass

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `lib/insecurity.ts:54`

How:

1. Upgrade express-jwt to 8.x: `npm install express-jwt@latest`
2. Upgrade jsonwebtoken to 9.x: `npm install jsonwebtoken@latest`
3. Enforce algorithm: `expressJwt({ secret: publicKey, algorithms: ['RS256'] })`
4. Rotate the RSA key pair and store private key in environment variable, not source

```
import { expressjwt } from 'express-jwt';
const isAuthorized = () => expressjwt({ secret: process.env.JWT_PUBLIC_KEY, algorithms: ['RS256'] });
```

Reference: https://github.com/auth0/express-jwt/releases

M-007 — Set noent:false in libxmljs2 parsing to disable external entity resolution

Addresses:

- F-007 — XXE via XML file upload with external entity resolution - XXE via XML file upload with external entity resolution

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/fileUpload.ts:83`

How:

1. Change `parseXml` options to remove `noent:true`
2. Use: `libxml.parseXml(data, { noblanks: true, noent: false, nocdata: true })`

```
const xmlDoc = vm.runInContext('libxml.parseXml(data, { noblanks: true, noent: false, nocdata: true })',
        sandbox, { timeout: 2000 })
```

Reference: https://`cwe.mitre.org/data/definitions/611.html`

M-008 — Validate extracted paths against target directory before writing

Addresses:

- F-008 — Zip slip path traversal via unzipper 0.9.15 - Zip slip path traversal via unzipper 0.9.15

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/fileUpload.ts:42`

How:

1. Resolve target path and check it starts with the intended base directory
2. Reject entries with .. or absolute paths
3. Use a safe-extract library or add explicit check: if (!`absolutePath.startsWith(path.resolve('uploads/complaints/'))`) throw new `Error('Zip-slip')`

```
const targetDir = path.resolve('uploads/complaints/');
const absolutePath = path.resolve(targetDir, path.basename(entry.path));
if (!absolutePath.startsWith(targetDir)) { entry.autodrain(); return; }
```

Reference: https://snyk.io/research/zip-slip-vulnerability

M-009 — Implement URL allowlist or SSRF protection library for outbound fetch

Addresses:

- F-009 — Server side request forgery via profile image URL upload - Server side request forgery via profile image URL upload

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/profileImageUrlUpload.ts:24`

How:

1. Validate URL against an allowlist of trusted image CDN domains
2. Block private IP ranges (10.x, 172.16.x, 192.168.x, 169.254.x, 127.x)
3. Use ssrf-req-filter or similar library

```
const allowedDomains = ['gravatar.com', 'avatars.githubusercontent.com'];
const urlObj = new URL(url);
if (!allowedDomains.includes(urlObj.hostname)) throw new Error('Domain not allowed');
```

Reference: https://`cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html`

M-010 — Add authentication middleware to sensitive file serving routes

Addresses:

- F-010 — Sensitive files and logs exposed without authentication - Sensitive files and logs exposed without authentication

Priority: P2 - This Sprint · **Effort:** Low · **File:** `server.ts:281`

How:

1. Add `security.isAuthorized()` middleware to `/support/logs/`
2. Add admin-only check to `/encryptionkeys/`
3. Remove or restrict `/ftp/` directory listing

```
// Remove directory-listing middleware; require auth on management endpoints.
// app.use('/ftp', serveIndex(...))   // delete
app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: Unauthenticated `GET /metrics` returns 401; `GET /ftp/` returns 404.

Reference: https://owasp.org/www-project-top-ten/2021/A01_2021-Broken_Access_Control

---

M-011 — Validate basket ownership against authenticated user's ID from JWT

Addresses:

- F-011 — IDOR basket accessible by any authenticated user - IDOR basket accessible by any authenticated user

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/basket.ts:19`

How:

1. Extract user ID from JWT: const `userId` = req.user.data.id
2. Add WHERE UserId = `userId` to basket query
3. Return 403 if basket does not belong to user

```
const userId = (req as any).user.data.id;
const basket = await BasketModel.findOne({ where: { id, UserId: userId }, include: [ProductModel] });
```

Reference: https://owasp.org/www-project-top-ten/2021/A01_2021-Broken_Access_Control

---

M-015 — Whitelist permitted layout file names; use path.basename() to strip directory traversal

Addresses:

- F-015 — Path traversal in data erasure layout parameter - Path traversal in data erasure layout parameter

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/dataErasure.ts:69`

How:

1. Validate layout against a whitelist of permitted values

2. Or use `path.basename(req.body.layout)` and prepend a fixed base directory
3. Verify resolved path starts with expected base directory

```
const allowed = ['gdpr.html', 'ccpa.html'];
if (!allowed.includes(req.body.layout)) return res.status(400).send('Invalid layout');
```

Reference: https:// `cwe.mitre.org/data/definitions/22.html`

M-016 — Add express-rate-limit to login endpoint (already in dependencies)

Addresses:

- F-016 — Missing rate limiting on authentication endpoint - Missing rate limiting on authentication endpoint

Priority: P2 - This Sprint · **Effort:** Low · **File:** `server.ts:594`

How:

1. Add `rateLimit({ windowMs: 15 * 60 * 1000, max: 10 })` to `POST /rest/user/login`
2. Consider CAPTCHA after 5 failed attempts
3. Add account lockout after N failures

```
const loginLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 10, message: 'Too many login
        attempts' });
app.post('/rest/user/login', loginLimiter, login());
```

Reference: https:// `cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html` #account-lockout

M-017 — Always derive UserId from verified JWT payload, never from request body

Addresses:

- F-017 — Mass assignment wallet balance manipulable via req.body.UserId - Mass assignment wallet balance manipulable via req.body.UserId

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/wallet.ts:12`

How:

1. Use req.body.UserId only when it comes from `appendUserId()` middleware which derives it from JWT
2. Verify that `appendUserId()` correctly sets req.body.UserId from JWT before these routes
3. Remove any path where client-controlled UserId is accepted

```
// Whitelist mutable fields server-side; never trust the body.
const ALLOWED = ['email', 'username'] as const
const patch = Object.fromEntries(
  Object.entries(req.body).filter(([k]) => ALLOWED.includes(k as any))
)
await user.update(patch)
```

Verification: POST `{ "role": "admin" }` to the register/update endpoint; confirm the stored row keeps the default role.

Reference: https://owasp.org/www-project-top-ten/2021/A01_2021-Broken_Access_Control

---

M-021 — Remove bypassSecurityTrustHtml; render IP as plain text

Addresses:

- F-021 — XSS last login ip rendered as trusted HTML - XSS last login ip rendered as trusted HTML

Priority: P2 - This Sprint · **Effort:** Low · **File:** `frontend/src/app/last-login-ip/last-login-ip.component.ts:39`

How:

1. Remove `bypassSecurityTrustHtml` wrapper
2. Use {{ `lastLoginIp` }} in template instead of [`innerHTML`]
3. Validate IP format on server before storing

```
// Never call bypassSecurityTrust*; let Angular sanitize.
// template:  <div [innerHTML]="product.description"></div>
// component: no DomSanitizer.bypassSecurityTrustHtml(...)
this.product.description = raw  // bound directly; Angular escapes
```

Verification: Insert `<img src=x onerror=alert(1)>` via the affected field and confirm the browser renders the escaped text, not an alert dialog.

Reference: https://angular.io/guide/security

M-022 — Store JWT in HttpOnly cookie to prevent XSS-based theft

Addresses:

- F-022 — Client side JWT stored in localStorage accessible to XSS - Client side JWT stored in localStorage accessible to XSS

Priority: P2 - This Sprint · **Effort:** High · **File:** `frontend/src/app/Services/authentication.service.ts:1`

How:

1. Set JWT as HttpOnly, SameSite=Strict, Secure cookie on server side
2. Update frontend to not read/write JWT from/to `localStorage`
3. Update API calls to use cookie-based auth (browser sends automatically)

```
// Move the JWT out of localStorage into an httpOnly cookie.
res.cookie('session', token, {
  httpOnly: true, secure: true, sameSite: 'lax', maxAge: 3600_000
})
```

Verification: Open browser DevTools, run `localStorage.getItem('token')` and confirm `null`.

Reference: https:// `cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html`

M-023 — Enforce authorization server-side; client guards are UI-only

Addresses:

- F-023 — Angular route guards bypassable client side - Angular route guards bypassable client side

Priority: P2 - This Sprint · **Effort:** Low · **File:** `frontend/src/app/app.guard.ts:12`

How:

1. Treat Angular guards as UI-only navigation helpers, not security controls
2. Enforce all authorization server-side with `isAuthorized()` and `isAccounting()` middleware
3. Document that client-side guards are not security boundaries

Reference: https:// `cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html`

M-025 — Sanitize product descriptions server-side; use textContent for display

Addresses:

- F-025 — Product description and review rendered as HTML stored XSS vector - Product description and review rendered as HTML stored XSS vector

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `frontend/src/app/search-result/search-result.component.ts:132`

How:

1. Remove `bypassSecurityTrustHtml` in `search-result.component.ts:132`
2. Sanitize product description HTML on creation/update server-side
3. Use `allowedTags` whitelist with sanitize-html (updated version)

```
// Never call bypassSecurityTrust*; let Angular sanitize.
// template:  <div [innerHTML]="product.description"></div>
// component: no DomSanitizer.bypassSecurityTrustHtml(...)
this.product.description = raw  // bound directly; Angular escapes
```

Verification: Insert `<img src=x onerror=alert(1)>` via the affected field and confirm the browser renders the escaped text, not an alert dialog.

Reference: https:// `cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html`

M-026 — Replace $where JavaScript expression with standard MarsDB equality query

Addresses:

- F-026 — NoSQL injection via $where operator in MarsDB reviews - NoSQL injection via $where operator in MarsDB reviews

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/showProductReviews.ts:36`

How:

1. Replace `db.reviewsCollection.find({ $where: 'this.product == ' + id })` with db. `reviewsCollection` . `find({ product: parseInt(id) })`
2. Validate id is a numeric integer before use

```
const productId = parseInt(req.params.id);
if (isNaN(productId)) return res.status(400).json({error: 'Invalid product ID'});
const reviews = await db.reviewsCollection.find({ product: productId });
```

Reference: https:// `cwe.mitre.org/data/definitions/943.html`

M-028 — Exclude password and totpSecret from all API serialization

Addresses:

- F-028 — User model returns sensitive fields in API responses (password hash, totpSecret) - User model returns sensitive fields in API responses (password hash, totpSecret)

Priority: P2 - This Sprint · **Effort:** Low · **File:** `models/user.ts:28`

How:

1. Add attributes: `{ exclude: ['password', 'totpSecret'] }` to all User queries
2. Or override `toJSON()` in the User model to strip sensitive fields
3. Ensure all user-returning endpoints explicitly select only needed columns

```
UserModel.findOne({ where: { id }, attributes: { exclude: ['password', 'totpSecret', 'deletedAt'] } })
```

Reference: https://`cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html`

M-030 — Mitigate SQL injection (T-004) eliminates schema exposure

Addresses:

- F-030 — Database schema exposure via SQL injection in search route - Database schema exposure via SQL injection in search route

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/search.ts:47`

How:

1. Fixing T-004 — SQL injection product search (UNION + schema exfil) (SQL injection in search) eliminates this exposure
2. Remove the sqlite_master query if the challenge is disabled

```
// Remove directory-listing middleware; require auth on management endpoints.
// app.use('/ftp', serveIndex(...))   // delete
app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: Unauthenticated `GET /metrics` returns 401; `GET /ftp/` returns 404.

Reference: https://`cwe.mitre.org/data/definitions/200.html`

M-031 — Fix NoSQL injection T-026 to eliminate this DoS vector

Addresses:

- F-031 — JavaScript injection via $where can crash MarsDB process - JavaScript injection via $where can crash MarsDB process

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/showProductReviews.ts:36`

How:

1. Mitigating T-026 — NoSQL injection via $where operator in MarsDB reviews (replace $where with equality query) eliminates this DoS vector

Reference: https://`cwe.mitre.org/data/definitions/400.html`

P3 — Next Quarter

M-012 — Use exact hostname matching instead of prefix matching for redirect allowlist

Addresses:

- F-012 — Open redirect bypass via allowlist prefix check - Open redirect bypass via allowlist prefix check

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/redirect.ts:19`

How:

1. Parse URL with new `URL(toUrl)` and compare only the origin/hostname
2. Maintain an allowlist of permitted hostnames, not URL prefixes

```
const allowed = ['www.paypal.com', 'github.com'];
try { const parsed = new URL(toUrl); if (!allowed.includes(parsed.hostname)) return res.status(406); }
        catch { return res.status(400); }
```

Reference: https:// `cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html`

M-013 — Add authentication or restrict /metrics to internal network

Addresses:

- F-013 — Prometheus metrics endpoint exposed without authentication - Prometheus metrics endpoint exposed without authentication

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:718`

How:

1. Add IP-based restriction or authentication check before metrics endpoint
2. Use express-ipfilter to restrict to monitoring subnet

```
// Remove directory-listing middleware; require auth on management endpoints.
// app.use('/ftp', serveIndex(...))   // delete
app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: Unauthenticated `GET /metrics` returns 401; `GET /ftp/` returns 404.

Reference: https://prometheus.io/docs/prometheus/latest/configuration/configuration/

M-014 — Restrict log access; send logs to separate log aggregation system

Addresses:

- F-014 — Access logs exposed to unauthenticated users tamper potential - Access logs exposed to unauthenticated users tamper potential

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:282`

How:

1. Remove `/support/logs` route entirely or add admin authentication
2. Forward logs to external SIEM (e.g. CloudWatch, Datadog, ELK)

```
// Log authn / authz outcomes with stable correlation ids.
logger.info({ event: 'auth.login.fail', userId, ip: req.ip, reqId })
logger.info({ event: 'authz.deny', userId, route: req.path, reqId })
```

Verification: A failed admin probe leaves an `authz.deny` line in the central log within 1 s.

Reference: https://owasp.org/www-project-top-ten/2021/A09_2021-Security_Logging_and_Monitoring_Failures

---

M-018 — Remove errorhandler in production; use generic error responses

Addresses:

- F-018 — Error handler reveals internal stack traces - Error handler reveals internal stack traces

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:1`

How:

1. Add NODE_ENV check: if (`process.env.NODE_ENV` !== 'development') use generic error handler
2. Return generic 500 message in production
3. Log details server-side only

Reference: https://
`cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html`

M-024 — Validate postMessage origin before processing; add frame-ancestors CSP directive

Addresses:

- F-024 — XSS via postMessage and iframe DOM based attack - XSS via postMessage and iframe DOM based attack

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `frontend/src/hacking-instructor/challenges/reflectedXss.ts:77`

How:

1. Validate event.origin against a known allowlist in all `postMessage` handlers
2. Add frame-ancestors 'self' to CSP header

```
// Never call bypassSecurityTrust*; let Angular sanitize.
// template:  <div [innerHTML]="product.description"></div>
// component: no DomSanitizer.bypassSecurityTrustHtml(...)
this.product.description = raw  // bound directly; Angular escapes
```

Verification: Insert `<img src=x onerror=alert(1)>` via the affected field and confirm the browser renders the escaped text, not an alert dialog.

Reference: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

---

M-029 — Verify review author matches authenticated user before allowing update

Addresses:

- F-029 — Product reviews allow unauthorized update IDOR in review update - Product reviews allow unauthorized update IDOR in review update

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/updateProductReviews.ts:1`

How:

1. Fetch review by id first
2. Check review.author === req.user.data.email
3. Return 403 if not the author

```
// Ownership check before touching a resource.
const basket = await Basket.findByPk(req.params.id)
if (!basket || basket.UserId !== req.user.id) return res.status(403).end()
```

Verification: Authenticate as user A; request `/api/Baskets/<B's id>` and confirm 403.

Reference: https://owasp.org/www-project-top-ten/2021/A01_2021-Broken_Access_Control

P4 — Backlog

No P4 mitigations.

10. Out of Scope

The following items are **explicitly excluded** from this threat model. Findings against these areas should be tracked separately.

- Third-party hosted dependencies and SaaS endpoints
- Browser runtime vulnerabilities and end-user device security
- Operating system kernel and container runtime
- Underlying network infrastructure (DNS, BGP, ISP)
- Physical security of hosting facilities

Appendix: Run Statistics

Field	Value
Invocation	`(not recorded)`
Generated	2026-05-30 15:28 UTC
Mode	full
Assessment depth	standard
Plugin version	0.4.0-beta (analysis v2)
Orchestrator model	claude-sonnet-4-6
Repository	`/home/mrohr/juice-shop`
Output directory	`/home/mrohr/juice-shop/docs/security`
Total analysis duration	39m 21s

Per-Stage Breakdown

Stage	Description	Agent	Model	Duration	Tool calls	Tokens
1	Threat Analysis & Triage	appsec-threat-analyst	claude-sonnet-4-6	25m 28s	153	190,554
2	Report Rendering	appsec-threat-renderer	claude-sonnet-4-6	8m 46s	48	99,664
3	Re-Render Loop (REPAIR_MODE)	appsec-threat-analyst	claude-sonnet-4-6	5m 01s	11	91,475
3	QA Review	`qa_checks.py`	none	0m 05s	0	0
Total	-	-	-	**39m 21s**	**212**	**381,693**

Per-Phase Duration Breakdown

Phase	Description	Agent (Model)	Duration
Phase 1	Context Resolution complete (parallel with Phase 2)	threat-analyst (sonnet-4-6)	2m 32s
Phase 2	Reconnaissance - recon-summary ready	recon-scanner (sonnet-4-6)	5m 58s
Phase 3	Architecture Modeling - 4 diagrams produced	threat-analyst (sonnet-4-6)	1m 10s
Phase 4	Attack Walkthroughs - 3 walkthroughs rendered	threat-analyst (sonnet-4-6)	(inline)
Phase 5	Asset Identification - 10 assets catalogued	threat-analyst (sonnet-4-6)	1m 30s
Phase 6	Attack Surface Mapping - 14 entry points (8 unauthenticated)	threat-analyst (sonnet-4-6)	1m 30s
Phase 7	Trust Boundary Analysis - 6 boundaries	threat-analyst (sonnet-4-6)	1m 30s
Phase 8	Security Controls - adequate=0 partial=4 weak=4 missing=8 unsafe=7	threat-analyst (sonnet-4-6)	1m 44s
Phase 9	STRIDE Enumeration - 31 threats (Critical: 9, High: 16, Medium: 6, Low: 0)	Nx stride-analyzer (sonnet-4-6)	7m 09s
Phase 10	Scan Synthesis - 2 secrets (from recon), 7 SCA/known-bad findings	threat-analyst (sonnet-4-6)	43s
Phase 10b	Triage Validation - 0 flags (0 warnings, 0 info)	appsec-triage-validator (sonnet-4-6)	1m 26s
Phase 11	Substeps 1-3 complete - need_render=true, Stage 2 ready	threat-analyst (sonnet-4-6)	1m 22s
Phase 11	Finalization (renderer mode)	threat-analyst (sonnet-4-6)	8m 28s

---

Appendix A — Vektor Taxonomy

This appendix defines the attacker-starting-position labels used in the Top Threats table and throughout §8 Threat Register. Each label answers the question what does the attacker need before the exploit begins?

Internet Anon

Attacker position: Unauthenticated attacker from the public internet · **Breach distance:** 1

Preconditions:

- Endpoint is reachable from the internet (no IP allowlist, no VPN)

- No authentication middleware blocks the request

Typical CWEs: CWE-89 · CWE-79 · CWE-306 · CWE-327 · CWE-611 · CWE-918

Typical OWASP Top 10: A01:2021, A03:2021, A07:2021

Internet User

Attacker position: Any authenticated low-privilege user (valid JWT / session) · **Breach distance:** 2

Preconditions:

- Attacker has signed up or otherwise obtained a valid user session
- Endpoint is behind auth but not behind role/admin checks

Typical CWEs: CWE-434 · CWE-611 · CWE-918 · CWE-352 · CWE-287

Typical OWASP Top 10: A01:2021, A04:2021, A05:2021, A10:2021

Internet Priv User

Attacker position: Authenticated admin-level user (JWT with admin role or equivalent) · **Breach distance:** 2

Preconditions:

- Attacker holds admin credentials or has elevated privileges
- Endpoint gated on admin role but still exploitable once reached

Typical CWEs: CWE-862 · CWE-79 · CWE-94

Typical OWASP Top 10: A01:2021

Victim-Required

Attacker position: Attacker needs victim interaction - social engineering, crafted link, or live session · **Breach distance:** 2

Preconditions:

- Victim must click a link, load a page, or have an active session
- Applies to XSS, CSRF, click-jacking, open redirect

Typical CWEs: CWE-79 · CWE-352 · CWE-601 · CWE-1021

Typical OWASP Top 10: A01:2021, A03:2021

Build-Time

Attacker position: Attacker controls a build input - CI runner, dependency, base image, or external data fetched during build · **Breach distance:** 3

Preconditions:

- Compromise of a dependency, registry, or base image
- OR compromise of a CI runner with write access to artifacts

Typical CWEs: CWE-506 · CWE-829 · CWE-1039 · CWE-1104

Typical OWASP Top 10: A08:2021

Repo-Read

Attacker position: Attacker gains read access to source repository (leaked clone, forked fork, insider, compromised developer workstation) · **Breach distance:** 3

Preconditions:

- Read access to the source tree at or after commit time
- No runtime exploit needed - the vulnerability is the content of the repo

Typical CWEs: CWE-798 · CWE-312 · CWE-540

Typical OWASP Top 10: A02:2021, A07:2021

n/a

Attacker position: Architectural / meta-finding - no runtime entry point, the finding describes a design defect aggregating multiple code-level findings

Preconditions:

- Finding is AF-NNN (architectural) rather than F-NNN (code-level)